

운영체제 보안 기말고사

9주차 강의 Virtual Memory

Memory Virtualization 메모리 가상화

OS(운영체제)는 물리적 메모리를 가상화한다.

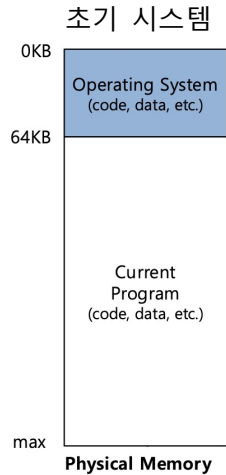
OS는 각 프로세스마다 가상(illusion)의 메모리 공간을 제공한다.

각 프로세스가 전체 메모리를 혼자 사용하는 것처럼 보이게 된다.

초기 시스템에서의 OS

메모리에 단 하나의 프로세스만 로드함.

낮은 이용률(Utilization)과 효율성(Efficiency)



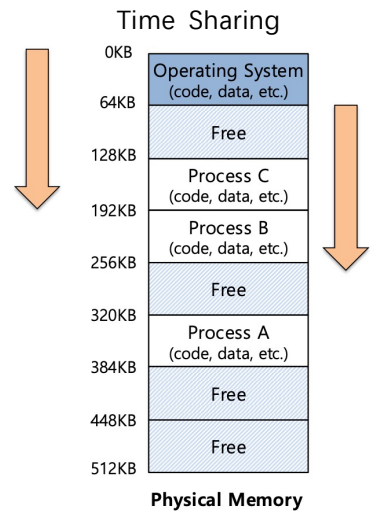
멀티프로그래밍과 시분할 Multiprogramming and Time Sharing

메모리에 여러 프로세스를 로드함.

하나를 짧은 시간 동안 실행함.

메모리 내에서 프로세스 간에 전환함.

이용률 Utilization과 효율성 Efficiency을 높임.



중요한 Protection Issue를 야기함.

다른 프로세스로부터의 잘못된(Errant) 메모리 접근

Address Space 주소 공간

OS는 물리적 메모리의 추상화를 만듭니다.

주소 공간은 실행 중인 프로세스에 대한 모든 것을 포함합니다.

그것은 프로그램 코드, 힙, 스택 등으로 구성되어 있습니다.

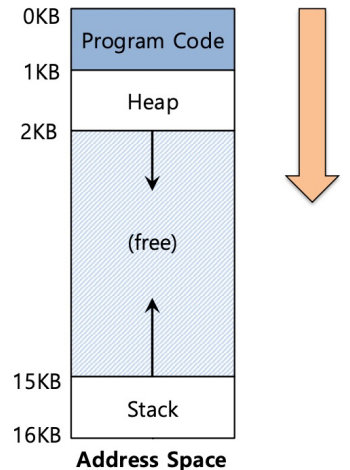
Code 코드 Instruction 명령어들이 존재하는 곳

Data 데이터 global 변수, static 변수

Heap 힙 : 메모리를 동적으로 할당함. C에서 malloc, 객체지향 언어에서 new

Stack 스택: 리턴 주소나 값들을 저장.

루틴(함수)으로 전달되는 인자(아규먼트)들과 로컬 변수들을 포함



Hexadecimal Representation 16진수 표현 :

이 강의에서는 32bit machine를 사용, NOT 64bit machine

$2^{32} = 4,294,967,296 = 4096M = 4GB$ 주소

0x00000000 ~ 0xFFFFFFFF

Memory Virtualization의 목적

- 목표 1: 프로그래밍에서의 사용 편의성
 - 목표 2: 시간과 공간 측면에서의 메모리 효율성
 - 목표 3: OS뿐만 아니라 프로세스에 대한 격리 보장
- 다른 프로세스의 잘못된(Errant) 접근으로부터의 보호

Principle of Isolation 격리의 원칙

OS는 프로세스들을 서로 격리하여 한 앱이 다른 앱을 망가뜨리지 못하게 막아준다.

메모리 격리를 통해 실행 중인 프로그램이 하부 OS 자체를 침범하거나 망가뜨리는 것을 방지

Process가 곧 격리의 단위이며, 각자 독립된 Address Space과 Time Slice를 할당

Microkernel : OS내부끼리 벽을 쳐서 서로 차단하는 구조

Mac OS, Win NT

필수적인 핵심 기능만 커널에 남기고, 나머지는 커널 외부로 분리
격리가 잘 되고 안정성이 높음

Monolithic Kernel : 유저 앱을 제외한 나머지 전부를 커널 영역에 통째로 두는 구조.

Linux

격리가 덜 되어 있거나 격리가 없음

Process

Process : unit of Isolation 격리의 단위

Thread : unit of execution 실행의 단위

Address Space : 각 프로세스가 침범하지 않도록 address space를 줌.

Execution Mode : User mode / Kernel Mode

Time Slice : 시분할 시스템에서 프로세스가 CPU를 사용할 수 있는 제한 시간. CPU 사용 상태

프로세스 구조: struct thread

pintos/src/threads/thread.h

```
struct thread
{
    /* Owned by thread.c. */
    tid_t tid;
    enum thread_status status;
    char name[16];
    /* Name (for debugging) */

    uint8_t *stack;
    /* Saved stack pointer. */
    int priority;
    /* Priority. */
    struct list_elem allelem;
    /* List element for all
    threads list. */

    /* Shared between thread.c and synch.c. */
    struct list_elem elem;
    /* List element. */

#ifdef USERPROG
    /* Owned by userprog/process.c. */
    uint32_t *pagedir;
    /* Page directory. */
#endif

    /* Owned by thread.c. */
    unsigned magic;
    /* Detects stack overflow. */
};
```

스레드라는 이름을 가지고 있지만 프로세스와 스레드를 둘 다 사용하는 방법.

pagedir : 메모리 관련된 것만 따로 떼어내서 프로세스를 사용하고 있음.

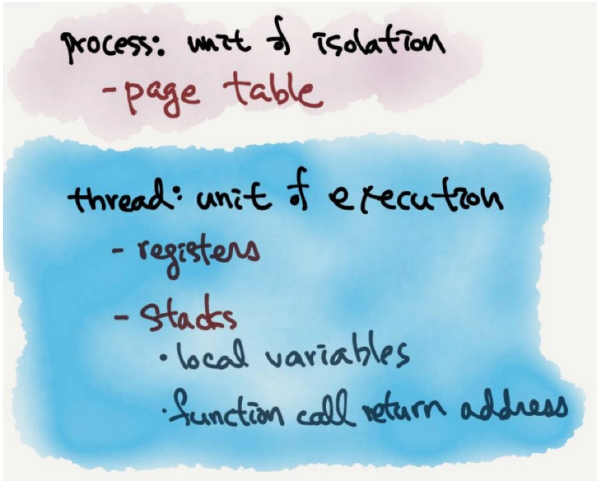
-> "가상 메모리가 아닌 실제 주소"가 저장되는 독립적인 페이지 테이블(pagedir)을 가짐으로써 프로세스로서의 격리 공간을 확보

Process VS thread

Process : 격리의 단위 - page table : 프로세스 구조체를 보면 프로세스마다 페이지 테이블이 있다.

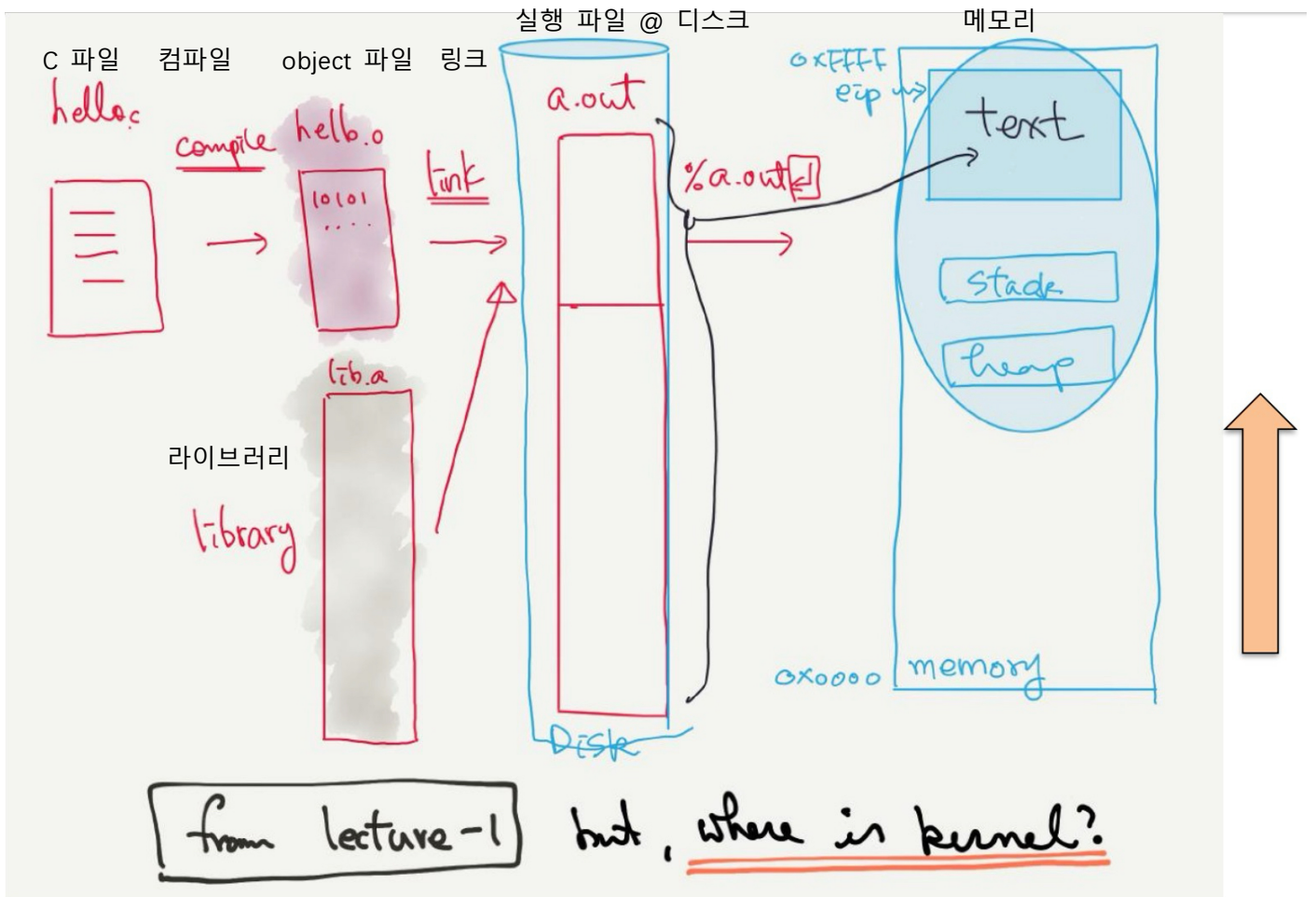
Thread : 실행의 단위

프로세스 : struct thread



스레드끼리의 메모리 공유
p->pagedir 페이지 테이블

스레드마다 다른 것
p->stack 스택
p->status 실행 상태
registers 레지스터



커널은 어디에 있지...?

Kernel = OS

eip : 프로그램 카운터. CPU가 실행해야 할 메모리 주소를 가리키는 레지스터.

로드된 가상 메모리의 text(코드) 영역 내부 주소를 조준하면서 프로세스의 명령어가 한 줄씩 실행됨

Virtual Address Space 가상 메모리 공간

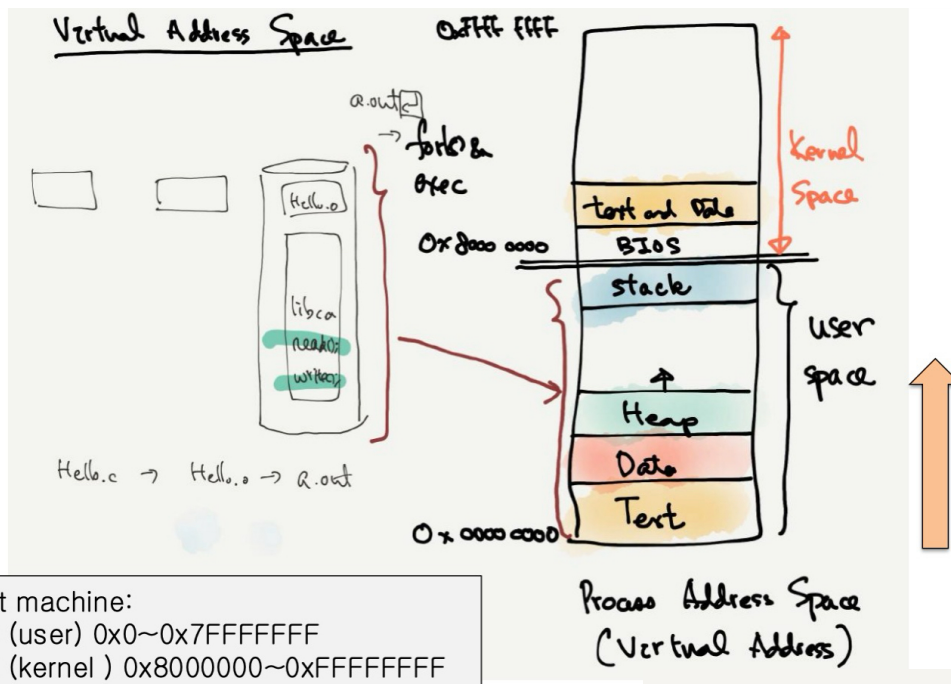
32bit machine: 하위는 유저가 사용하고 상위는 커널이 사용하는 형태

2GB (user) 0x0~0x7FFFFFFF

2GB (kernel) 0x80000000~0xFFFFFFFF

Hello.c → Hello.o → a.out

디스크에 보관되어 있던 a.out을 실행하는 순간, OS 내부에서는 fork() & exec 시스템 호출 메커니즘이 발동하여 user space에 새로운 Text, Data, Heap, stack 영역을 할당하고 프로그램을 메모리에 안착



User stack vs Kernel Stack

- 유저 모드에서 실행 중 - user stack
- 커널에서 실행 중 - kernel stack

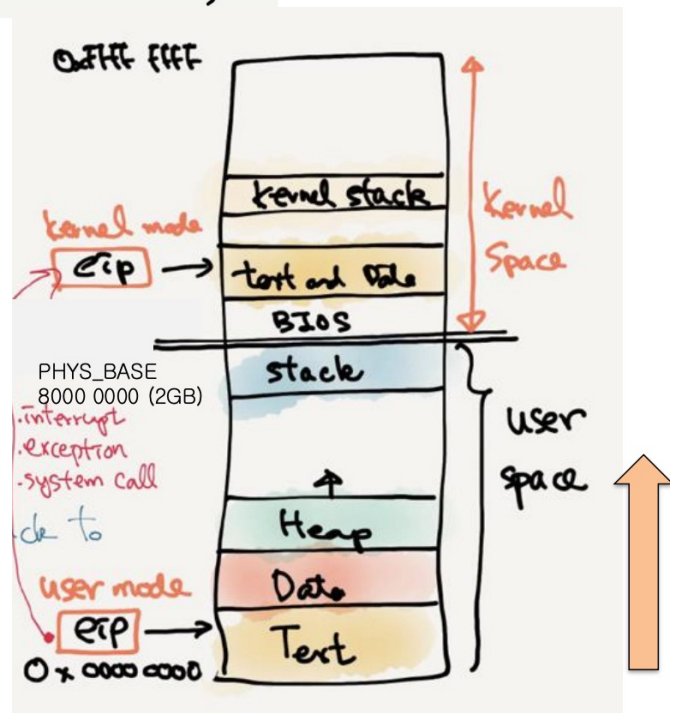
System call 시스템 콜

- 커널로 진입 -> 유저 스택에서 커널 스택으로 전환
- > privilege level 특권레벨 상승

interrupt, exception(0으로 나누기), system call 등의 이유로 kernel mode 진입

Virtual Address

실행 중인 프로그램 내부의 모든 주소는 가상 주소입니다.
 OS(운영체제)는 가상 주소를 물리 주소로 변환(매핑)합니다



EIP=PC in intel CPU

```
#include <stdio.h>
#include <stdlib.h>

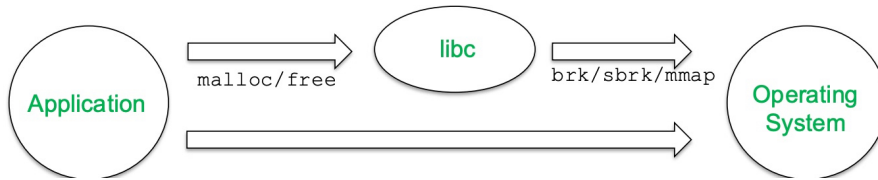
int main(int argc, char *argv[]){

    printf("location of code : %p\n", (void *) main);          main 함수의 주소 출력
    printf("location of heap : %p\n", (void *) malloc(1));    1바이트 할당해서 heap 주소 출력
    int x = 3;
    printf("location of stack : %p\n", (void *) &x);          지역변수 x의 주소를 출력

    return x;
}
```


10주차 강의 Address Translation

Virtual Address Space

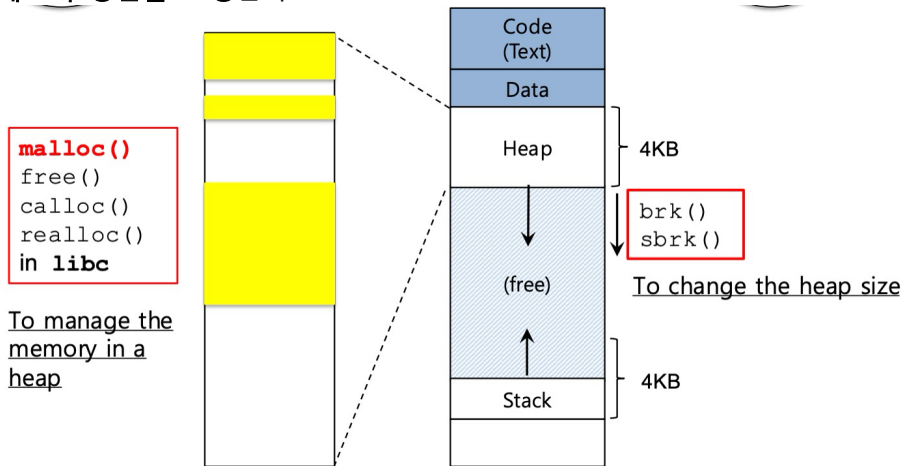


Application : 사용자 프로그램

libc : C 표준 라이브러리

Application이 메모리를 할당받거나 해제할 때 libc의 malloc/free 함수를 호출한다.

libc 내부의 힙 관리자가 가지고 있던 메모리가 부족해지면 OS는 brk, sbrk, mmap같은 시스템 콜을 보내 더 큰 메모리 공간을 요청한다.



malloc()

```
#include <stdlib.h>
```

```
void* malloc(size_t size)
```

힙(heap) 영역에 메모리 공간을 할당합니다.

Argument 인자

size_t size : 메모리 블록의 크기 (바이트 단위)

size_t는 unsigned int 타입입니다.

Return

성공 시 : malloc에 의해 할당된 메모리 블록을 가리키는 void 포인터

실패 시 : NULL 포인터

```
int *arr;  
arr=(int*)malloc(sizeof(int)*5);  
arr[0]=1  
arr[4]=20
```

Memory Virtualizaing with Efficiency and Control 효율성과 제어를 동반한 메모리 가상화

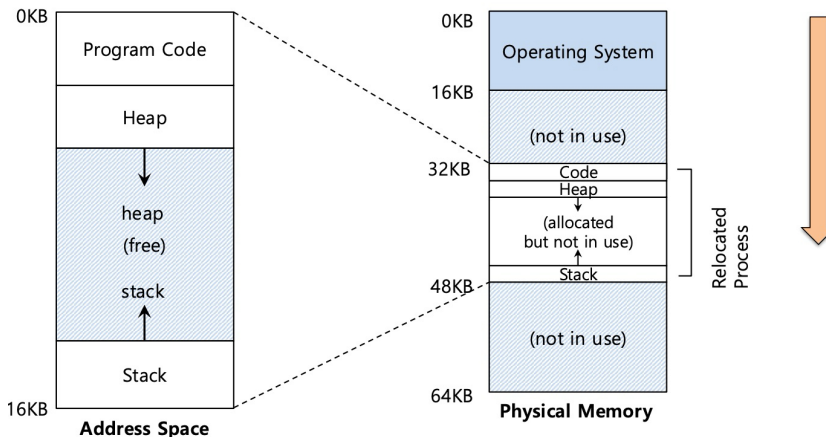
메모리 가상화는 효율성과 제어를 위해 Limited Direct Execution(LDE)이라고 알려진 유사한 전략을 취합니다.

메모리 가상화에서 efficiency과 control은 hardware support을 통해 달성됩니다.

e.g. 레지스터, Page-Table, TLB(Translation Look-aside Buffers)

Address Translation

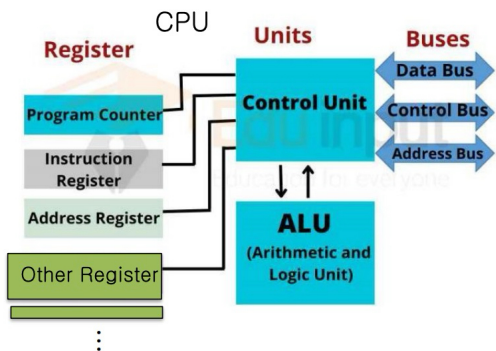
하드웨어는 **virtual address**를 실제 **physical address**로 변환합니다.
 프로세스가 원하는 실제 정보는 사실 physical address에 저장되어 있습니다.
 OS는 하드웨어를 설정하기 위해 **중요한 시점(key points)**마다 개입해야 합니다.
 OS는 신중하고 현명하게 개입하기 위해 **메모리를 관리**해야 합니다.



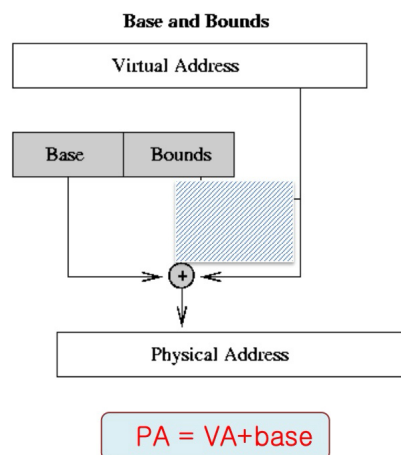
OS는 프로세스를 주소 0번지가 아닌, 실제 물리 메모리의 다른 어딘가에 배치하고 싶어 한다.

프로세스의 virtual address space는 항상 0번지부터 시작합니다.

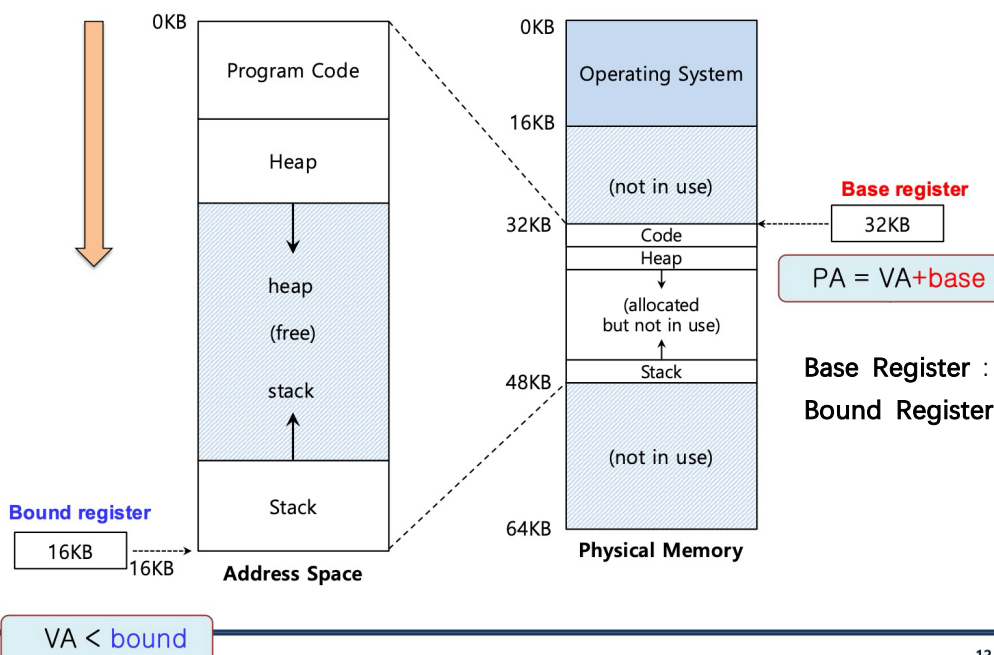
base 및 bound 레지스터



↑ 여기 초록색 부분에 Base와 Bound 레지스터가 위치한다.



VA가 base와 합쳐져
PA가 된다!



Base Register : 물리 메모리에서 시작점
Bound Register : 가상 주소의 크기(끝 점)

Dynamic Reallocation

프로그램이 실행되기 시작할 때, OS는 프로세스가 물리 메모리의 어디에 로드되어야 할지 결정한다.

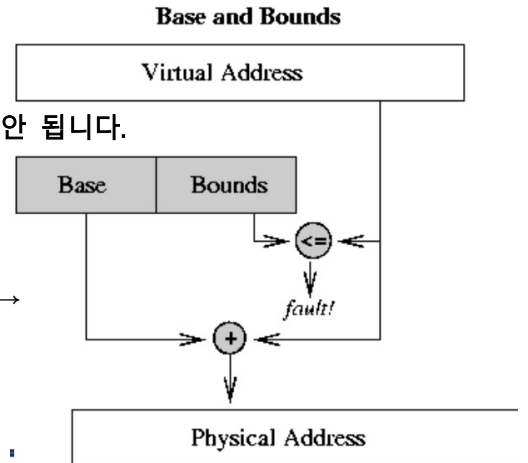
1. base 레지스터에 값을 설정합니다.

$$\text{physical address} = \text{virtual address} + \text{base}$$

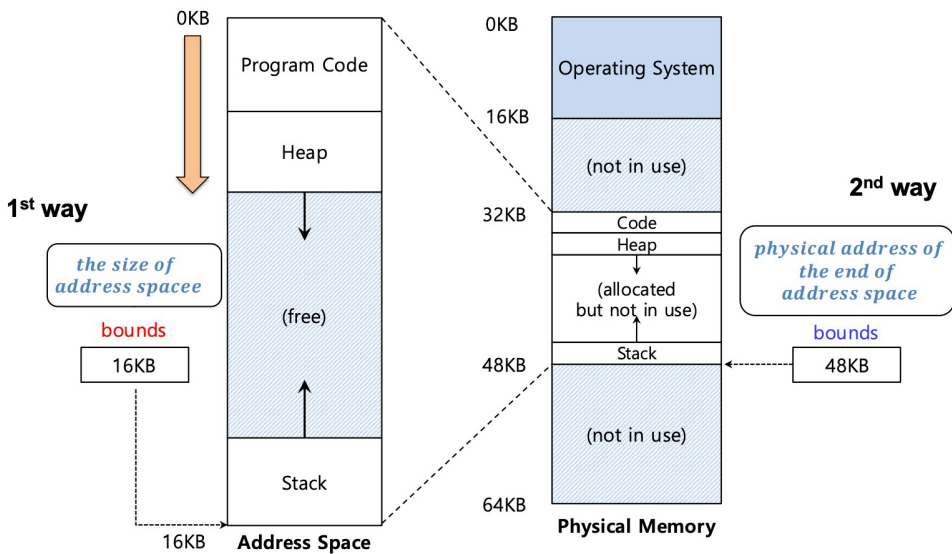
2. 모든 가상 주소는 bound보다 크거나 같아서는 안 되며, 음수여서는 안 됩니다.

$$0 \leq \text{virtual address} < \text{bound}$$

잘못된 곳에 접근하면 fault! →



Bound Register의 2가지 방식



1st : 가상 주소 공간의 크기 e.g. 16KB

2nd : 물리 메모리 상의 끝 주소 e.g. 48KB

메모리 가상화의 OS 이슈

OS는 base-and-bounds 접근 방식을 구현하기 위해 반드시 개입해야 한다.

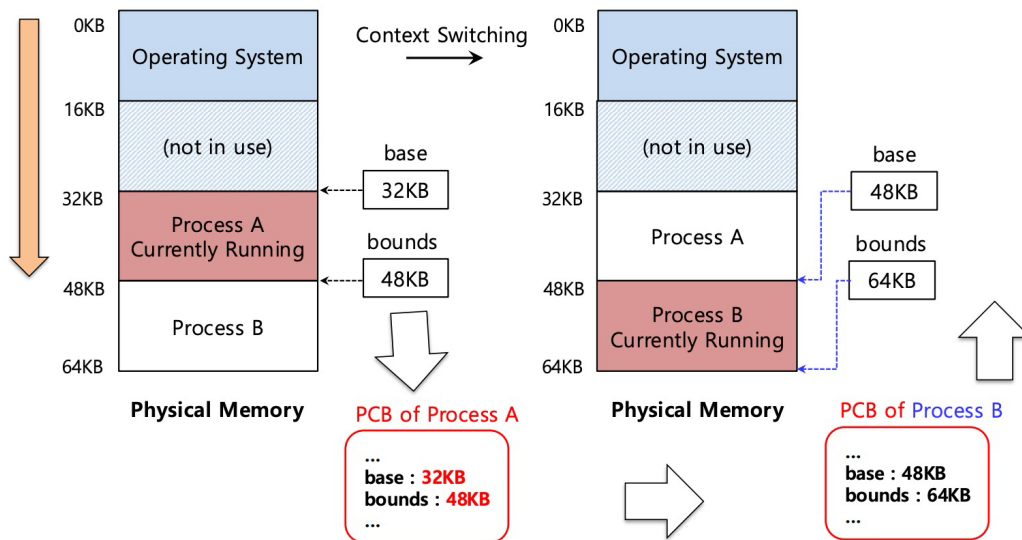
세 가지 결정적인 시점(타이밍):

1. 프로세스가 실행을 시작할 때
물리 메모리에서 프로세스의 가상 주소 공간을 배치할 빈 공간을 찾는 것.
2. 프로세스가 종료될 때
다른 용도로 재사용할 수 있도록 메모리를 회수하는 것
3. Context Switch가 발생할 때
base와 bound 레지스터 쌍(pair)을 저장하고 복원하는 것.

Context Switch가 발생할 때

OS는 base-and-bounds 레지스터 쌍을 반드시 저장하고 복원해야 합니다.

process struct 또는 PCB 내에 구조적으로 보관



Segmentation

Base-and-Bound 접근의 비효율성

엄청나게 큰 덩어리의 "빈(free)" 공간

이 "빈" 공간이 실제 물리 메모리를 그대로 차지하고 있다.. 낭비!

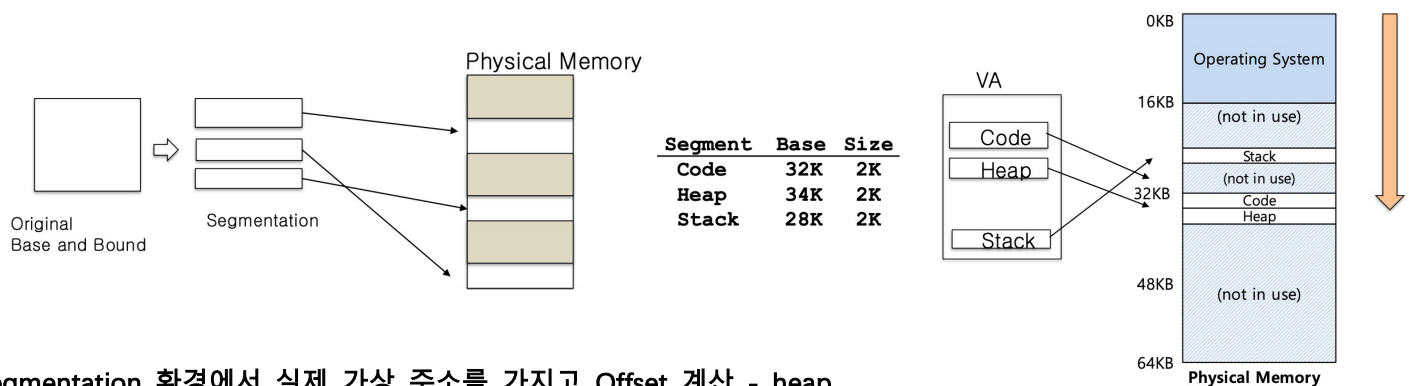
address space가 물리 메모리의 남은 공간에 맞지 않으면 프로그램을 실행하기 어렵다.

Segment : 특정 길이를 가진 address space의 연속적인 일부분일 뿐입니다.

논리적으로 구분되는 Segment : Code, Stack, Heap을 쪼개서 각각 넣기!

각 segment는 물리 메모리의 서로 다른 부분에 배치될 수 있다.

Base와 Bounds 레지스터가 각 세그먼트마다 각각 존재



Segmentation 환경에서 실제 가상 주소를 가지고 Offset 계산 - heap

$$\text{Offset} = \text{목표 VA}(4200) - \text{Segment 시작 주소}(4096) = 104$$

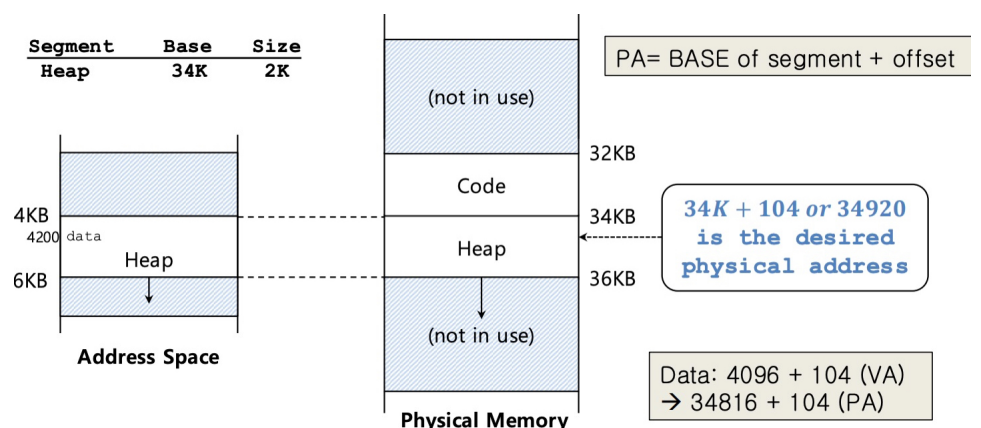
$$\text{PA} = \text{BASE of segment} + \text{offset}$$

$$\text{Data: } 4096 + 104 \text{ (VA)}$$

$$\rightarrow 34816 + 104 \text{ (PA)}$$

만약 7KB에 접근하려고 시도하면?

=> **Segmentation Fault!**

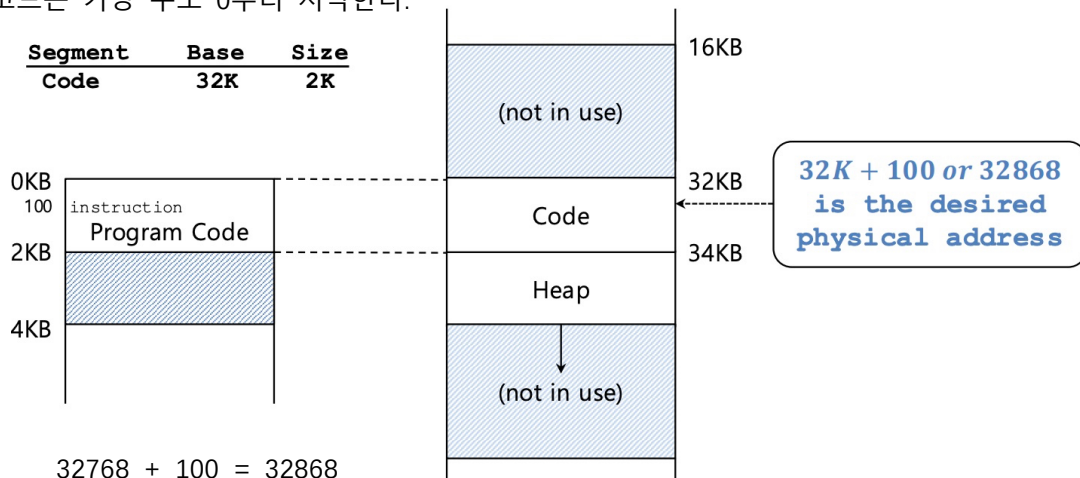


11주차 강의 Paging

Segmentation 환경에서 실제 가상 주소를 가지고 Offset 계산 - code

offset의 가상 주소 100의 오프셋은 100이다.

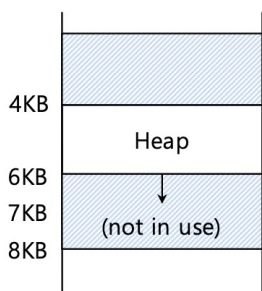
코드는 가상 주소 0부터 시작한다.



Segmentation Fault 또는 Violation

heap의 끝을 넘어선 7KB와 같은 illegal address가 참조되면, 운영체제는 segmentation fault를 발생시킨다.

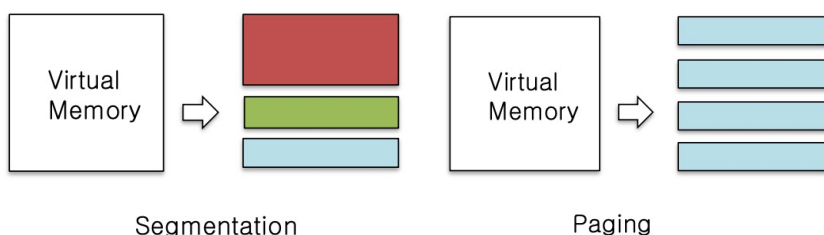
하드웨어(CPU/MMU)가 해당 주소가 out of bounds 된것을 감지



Address Space

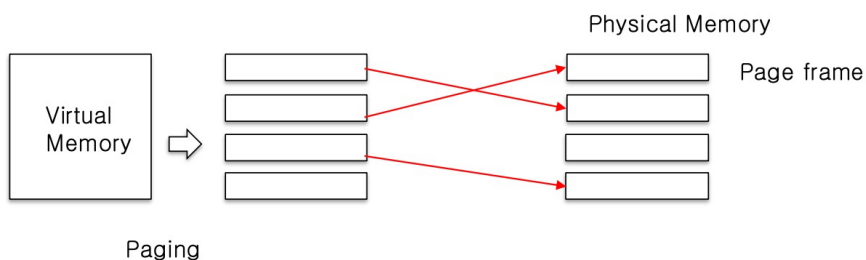
Paging 페이징

페이징은 주소 공간을 고정된 크기(fixed-sized)의 단위인 페이지(page)로 쪼갭니다.



Segmentation : 논리적 세그먼트(코드, 스택, 힙 등)의 가변적인 크기(variable size)

Paging은 실제 **physical memory** 또한 **page frame**이라고 불리는 동일한 크기의 페이지 조각들로 쪼개진다.



Page table for process A

Page (virtual)	Page Frame
1	50
2	21
3	70

Virtual Address를 Physical Address로 변환하기 위해 프로세스당 하나의 Page table이 필요합니다. ↑

Paging의 장점

Flexibility : 주소 공간의 abstraction을 효과적으로 지원함.

힙과 스택이 어떤 방향으로 자라나고 어떻게 사용되는지에 대한 구조적 가정이 전혀 필요 없음.

Simplicity : 빈 공간(free-space) 관리가 극도로 간편함.

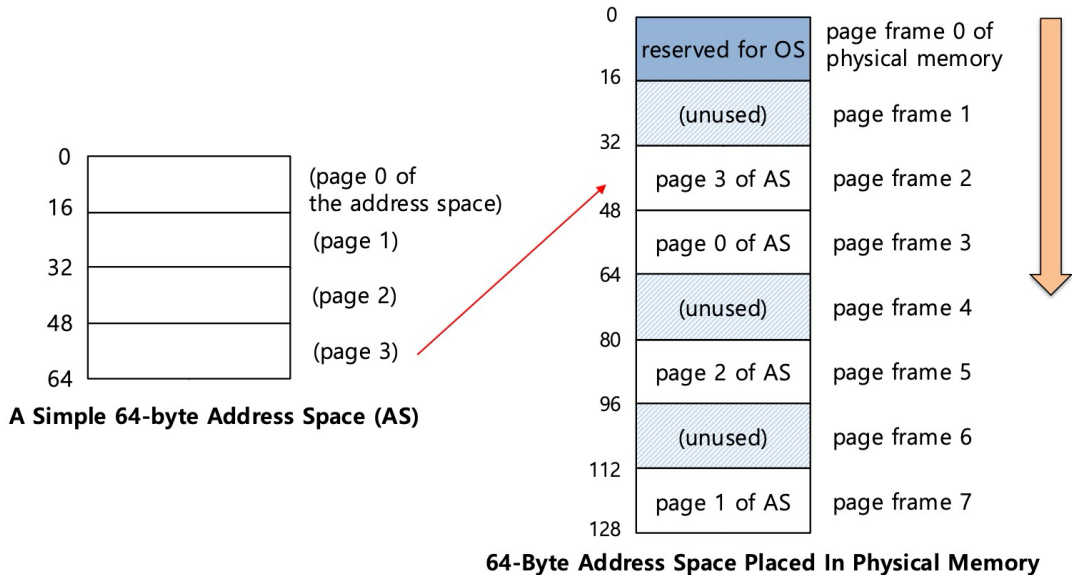
가상 주소 공간의 페이지와 물리 메모리의 페이지 프레임의 크기가 동일함.

free list를 할당하고 유지하기가 매우 쉬움

Simple Paging의 예제

16 바이트 크기의 페이지를 가진 64바이트 (가상)주소공간

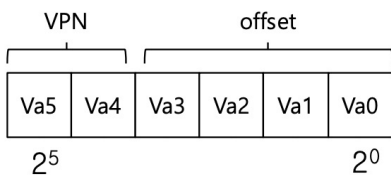
16 바이트 크기의 페이지 프레임들을 가진 128바이트 실제 물리 메모리



주소 해석(Paging)

VPN(Virtual Page Number) : 주소의 상위 비트에 위치하며, "몇 번째 가상 페이지인가?"를 가리킴

Offset : 주소의 하위 비트에 위치하며, "그 페이지 내부에서 정확히 몇 바이트만큼 떨어져 있는가?"



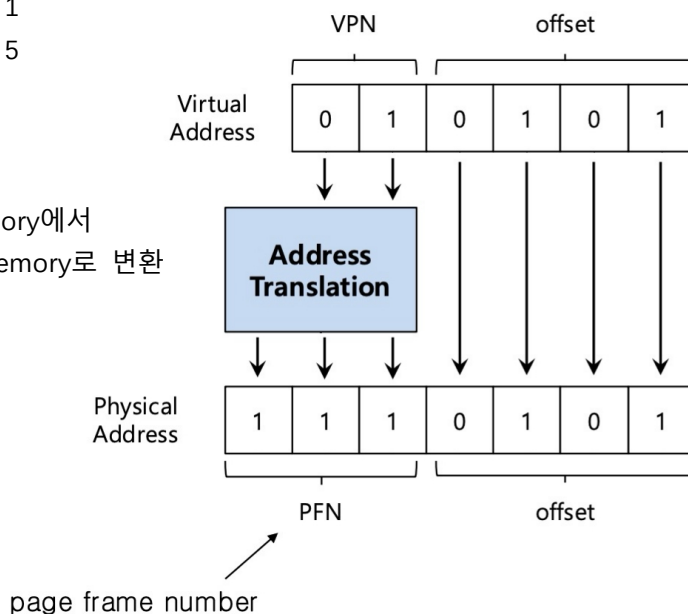
e.g. 64바이트 주소공간에서 Virtual Address 21 = 010101 이므로

VPN은 01 1

offset은 0101 5

64바이트 Virtual Memory에서
128바이트 Physical Memory로 변환

2자리에서 3자리로!



VA 21
→ VPN 1, offset 5

↓
PFN 7, offset 5
→ 117

Page table for process A

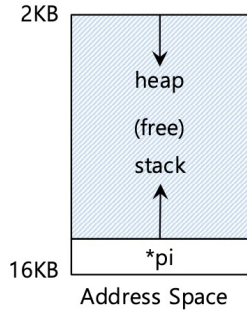
Page (virtual)	Page Frame
0	3
1	7
2	5
3	2

malloc() 복습

```
int *arr;
arr=(int*)malloc(sizeof(int)*5);
arr[0]=1
arr[4]=20
```

성공하면 포인터에 void 포인터 반환
실패하면 NULL 포인터 반환

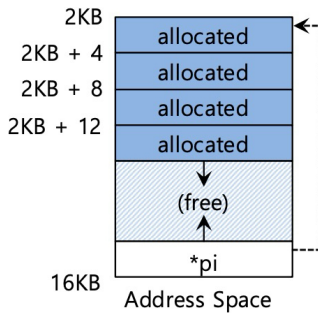
메모리 할당



-----> pointer

```
int *pi; // local variable
```

로컬 변수 자체는 Stack에 있다!



```
pi = (int *)malloc(sizeof(int)*4);
```

동적할당한 공간은 Heap에 쌓인다!

sizeof()

숫자를 직접 입력하는 대신, malloc에서 크기를 지정하기 위해 루틴과 매크로(sizeof)가 활용됩니다.
변수와 함께 sizeof를 사용할 때 나타나는 두 가지 유형의 결과

- The actual size of 'x' is known at run-time.

```
int *x = malloc(10 * sizeof(int));
printf("%d\n", sizeof(x));
```

4

x의 크기가 런타임에 정해짐

- The actual size of 'x' is known at compile-time.

```
int x[10];
printf("%d\n", sizeof(x));
```

40

x의 크기가 컴파일 타임에 정해짐

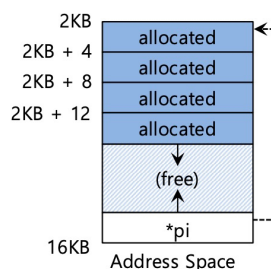
free()

```
#include <stdlib.h>
void free(void* ptr)
```

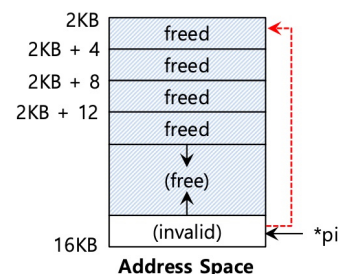
Argument : malloc으로 할당된 메모리 블록을 가리키는 포인터

Return : 없음

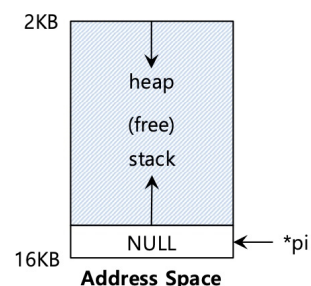
메모리 해제 freeing



```
pi = (int *)malloc(sizeof(int)*4);
```



```
free(pi);
```



```
pi=NULL
```

12주차 강의 Translation Lookaside Buffer

Pointer Variable 포인터 변수

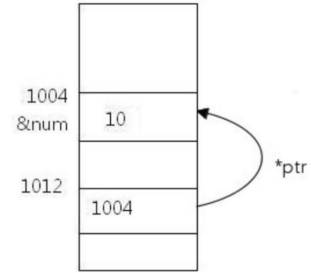
- Format: * + variable name ex) *p

```
int num = 10;
int *ptr;
ptr = &num;
```

```
*ptr=12
```

```
int *pi=(int*)malloc(sizeof(int)*4);
*pi=1
*(pi+1)=2
```

```
pi[0]=1
pi[1]=2
```

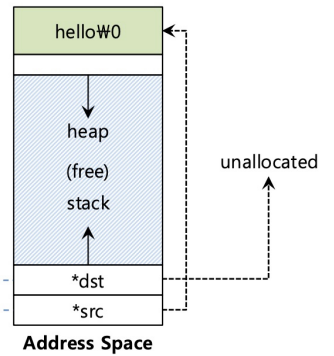


메모리 할당하는걸 잊으면?

잘못된 코드

```
char *src = "hello"; //문자열 상수
char *dst;           //할당안된 포인터변수
strcpy(dst, src);    //segfault 그리고 프로그램 죽음
```

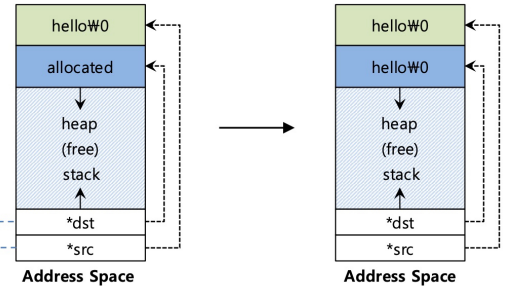
```
strcpy(dst, src);
```



올바른 코드

```
char *src = "hello"; //문자열 상수
char *dst = (char *)malloc(strlen(src) + 1 ); // 할당됨
strcpy(dst, src);    //정상적으로 작동
```

```
strcpy(dst, src);
```



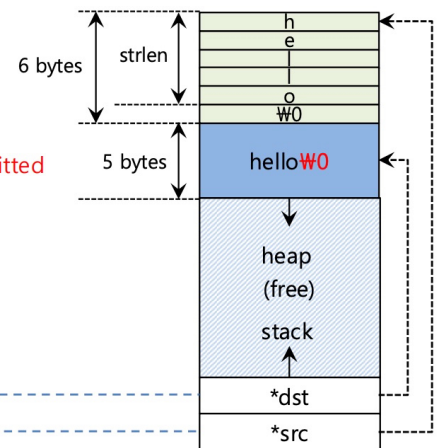
할당은 했는데 충분하지 않은 공간 할당하면?

```
char *src = "hello"; // 문자열 상수
char *dst = (char *)malloc(strlen(src)); // 너무 작게 할당됨 (오류 원인)
strcpy(dst, src); // (겉보기에는) 정상 작동함
```

null문자를 위한 공간을 할당하지 않았다!

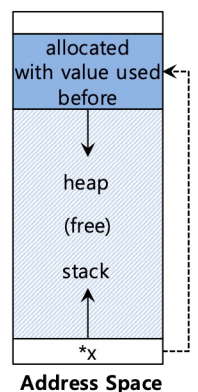
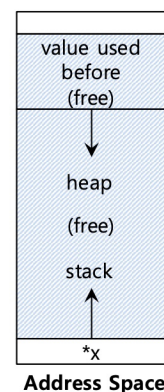
-> 오버플로우 발생 다른 공간을 침범할 수 있다.

```
strcpy(dst, src);
```



초기화하는걸 잊으면?

```
int *x = (int *)malloc(sizeof(int)); // 할당됨
printf("x = %d\n", *x); // 초기화되지 않은 메모리 접근
```

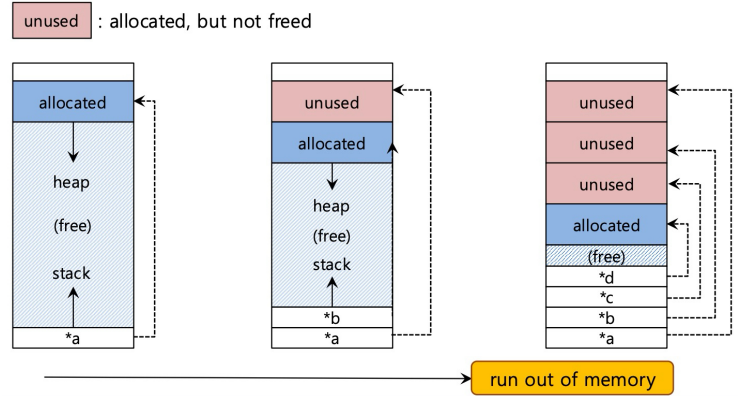


Memory Leak 메모리 누수

프로그램이 메모리를 해제(free)하지 않고 계속해서 할당(malloc)만 반복합니다.

프로그램의 메모리가 완전히 고갈되어(Runs out of memory), 결국 OS에 의해 강제 종료(Killed)당합니다.

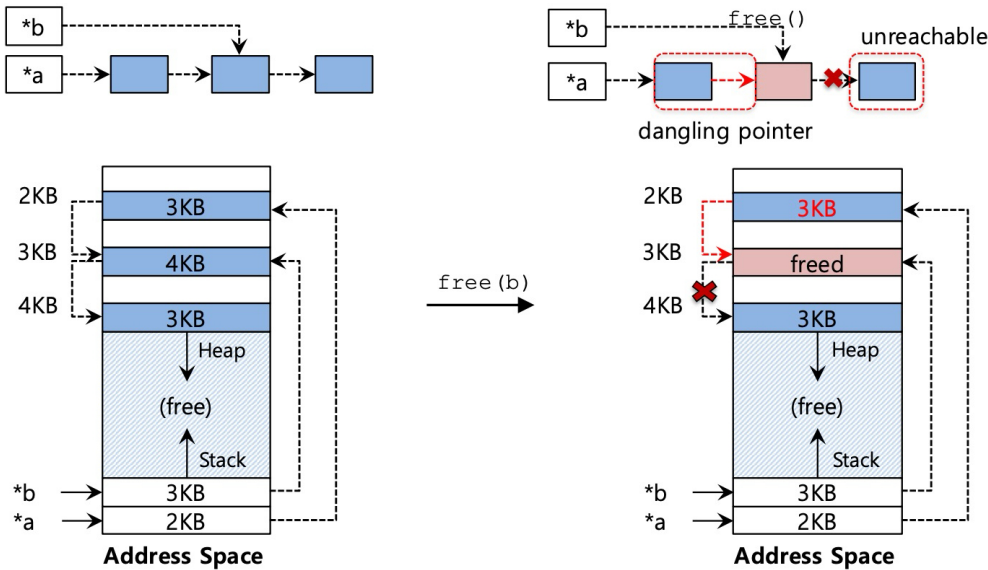
```
while(1)
    malloc(4) ;
```



Dangling Pointer 땡글링 포인터

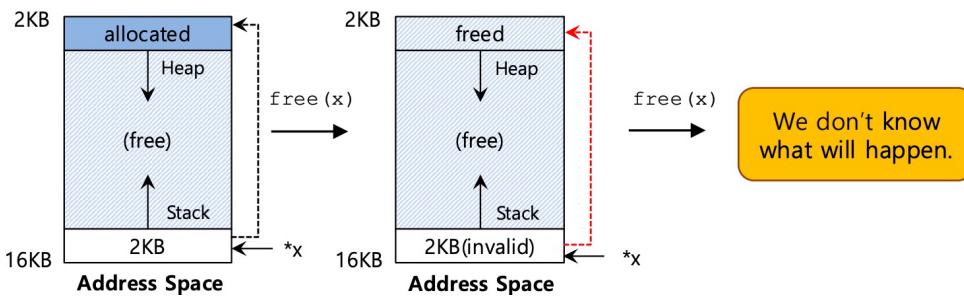
메모리가 아직 사용 중인데 Free되어버린 상태.

프로그램이 이미 free되어 invalid pointer를 가지고 메모리에 잘못 접근하게 됩니다.



잘못된 free()

```
int *x = (int *)malloc(sizeof(int));    // 메모리 할당
free(x);                                // 정상 해제
free(x);                                // 이미 죽은 녀석을 또 해제 - 에러!
```



malloc()을 통해 할당받지 않은 엉뚱한 메모리를 해제하는 경우.

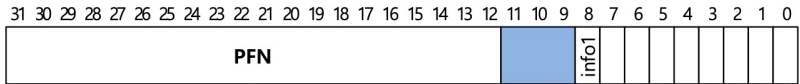
```
int *x = (int *)malloc(sizeof(int));    // 정상 할당
free(x + 12);                           // 엉뚱한 주소 해제 - 에러!
```

Page Table은 어디에 저장될까?

페이지 테이블은 대단히 커질 수 있다.
4-KB 페이지를 가진 32비트 주소 공간, VPN을 위한 20비트
4-KB 페이지에 대해서 나머지 12비트는 오프셋이 사용.

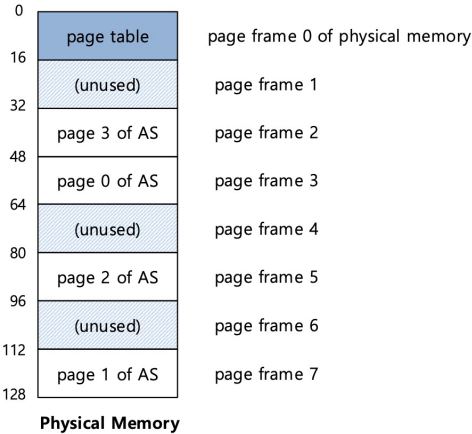
Page Table 전체 크기 4MB
4MB = 2²⁰ 개 Entry * Page Table Entry(PTE)당 4바이트

- * PTE (Page Table Entry): 가상 주소를 물리 주소로 변환하는 정보와 각종 상태 플래그를 담고 있는 Page Table의 개별 행 단위를 뜻합니다.
- * PFN (Page Frame Number) : 가상 페이지가 실제 물리 메모리에 매핑되어 들어가는 실제 프레임의 번호



Page Table Entry(PTE)

각 프로세스의 페이지 테이블은 메모리에 저장된다.
CPU에 넣기는 너무 크다!



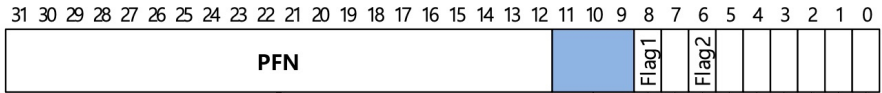
Kernel Phsical Memory 내의 페이지 테이블 예제 →→→

페이지 테이블은 user space가 아니라 kernel space에 저장되어 있다.

Page Table안에는 뭐가 있을까?

페이지 테이블은 가상 주소를 물리 주소로 매핑(대응)하는 데 사용되는 자료 구조다.
가장 간단한 형태: linear page table, array

OS는 VPN을 통해 배열의 인덱스를 지정하고, Page Table Entry를 조회한다.

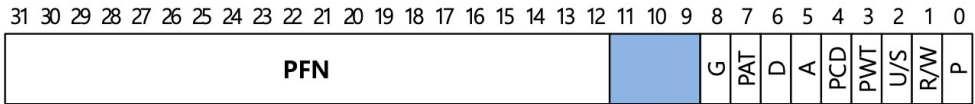


Page Table Entry(PTE)

PTE에서의 Common Flag(공통 플래그)

- Valid Bit : 특정 주소 변환이 유효한지 여부를 나타냄.
- Protection Bit (R/W bit) : 해당 페이지를 읽을 수 있는지, 쓸 수 있는지, 혹은 실행할 수 있는지 여부를 나타냄.
- Present Bit : 이 메모리가 물리 메모리에 있는지 아니면 디스크에 있는지 여부를 나타냄. 1이면 물리 메모리에 있다. 0이면 물리 메모리에 없다(디스크에 있음)
- Dirty Bit : 페이지가 메모리에 로드된 이후 수정되었는지 여부를 나타냄.
- Reference Bit : 페이지가 접근되었음을 나타냄.

x86 PTE 예제

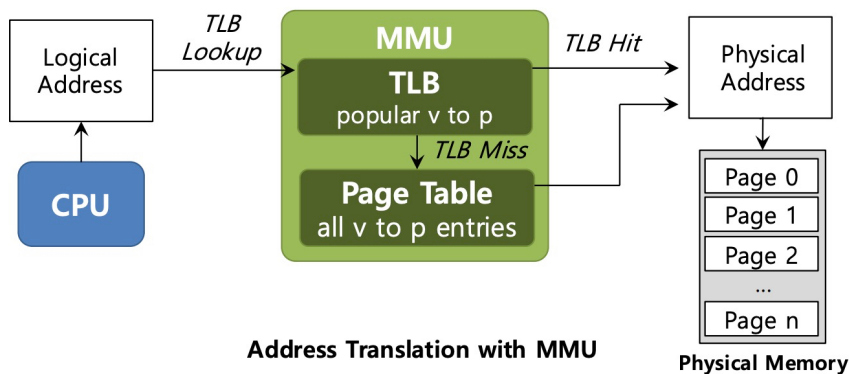


- P : Present : 지금 메모리에 있는가? 1이면 있음 0이면 없음
- D : Dirty Bit : 메모리에 올라온 이후 수정되었나?
- RW Read-only인가 RW인가? U/S 유저/관리자 페이지인가? A Reference bit PFN 물리 프레임 번호

TLB(Translation Lookaside Buffer)

칩의 메모리 관리 장치(MMU)의 일부분이다.

자주 쓰이는 가상 주소-물리 주소 변환 정보를 담은 Hardware Cache이다.



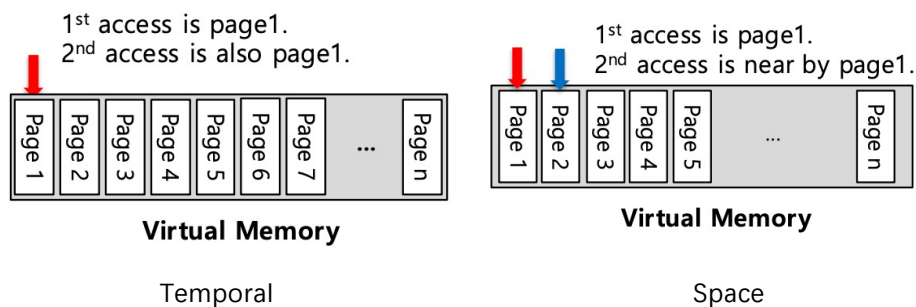
Locality

Temporal Locality 시간 지역성

최근에 접근된 명령어(인스트럭션)나 데이터 항목은 가까운 미래에 곧 다시 접근될 가능성이 높다.

Spatial Locality 공간 지역성

만약 프로그램이 주소 x에 있는 메모리에 접근한다면, 곧 x 근처에 있는 메모리에 접근할 가능성이 높다.



Physical Memory를 넘어서. 메커니즘

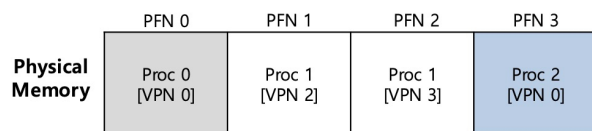
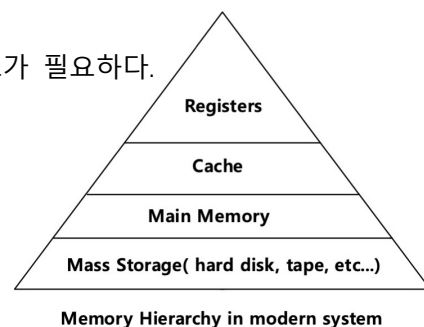
운영체제는 현재 자주 쓰이지 않는 주소 공간의 일부를 숨겨둘(저장해둘) 장소가 필요하다.

현대 시스템에서 이 역할은 보통 Hard Disk Drive에 의해 수행된다.

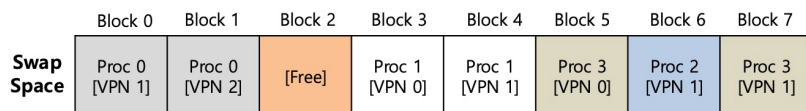
공간 SWAP

페이지들을 전후로 이동시키기 위해 디스크 상에 일정 공간을 예약(확보)한다.

OS는 페이지 크기 단위로 스왑 공간을 기억해야 한다.



프로세스 0번의 vpn 0번은 메모리에 있지만
프로세스 0번의 vpn 1, 2번은 swap space에 있습니다



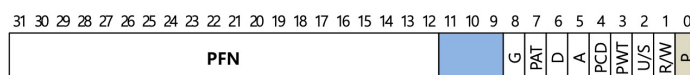
Physical Memory and Swap Space

Present Bit

페이지들을 디스크와 주고받는 Swapping을 지원하기 위해 시스템의 더 상위 계층에 어떤 메커니즘을 추가한다.

하드웨어가 PTE를 들여다볼 때, 해당 페이지가 물리 메모리에 존재하지 않는다는 것을 발견할 수 있다.

Value	Meaning
1	page is present in physical memory
0	The page is not in memory but rather on disk.



An x86 Page Table Entry(PTE)

15주차 강의 Translation Lookaside Buffer

Page Fault

physical memory에 없는 page에 접근하는 것이다.

만약 page가 존재하지 않고 swapped disk 상태라면, OS는 해당 page fault를 처리하기 위해 그 page를 memory로 다시 swap해 올 필요가 있다.

Page replacement

OS는 가져오려고 하는 새로운 page들을 위한 공간을 만들기 위해 기존 page들을 page out하는 것을 선호한다. 쫓아내거나 교체할 page를 고르는 과정은 **page-replacement policy**라고 알려져 있다.

FIFO, LRU, LFU 등등...

언제 Replacement를 할까?

Lazy approach

OS는 memory가 완전히 가득 찰 때까지 기다렸다가, 다른 어떤 page를 위한 공간을 만들 때만 page를 replace한다. -> 비현실적이다!

Swap Daemon/Page Daemon

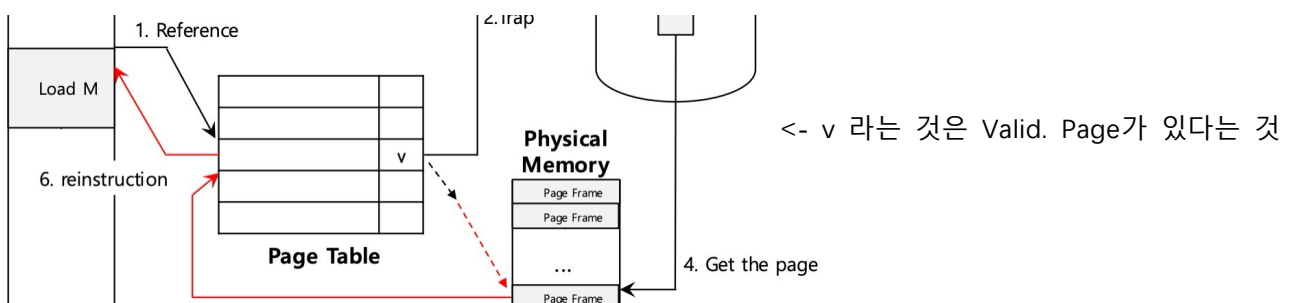
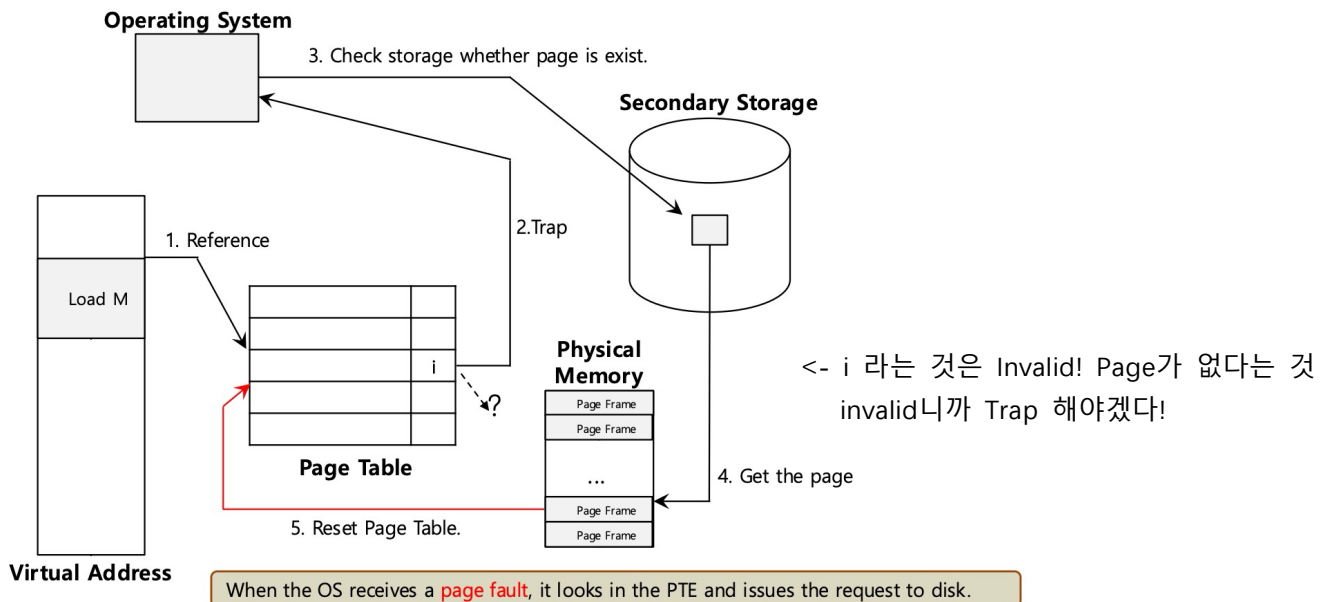
사용 가능한 page 수가 LW(low watermark, 여유 메모리의 하한)보다 적으면, memory를 비우는 것을 담당하는 **background thread**가 실행된다.

사용 가능한 page 수가 HW (high watermark, 여유 메모리의 상한선)만큼 있을 때까지 page들을 내쫓는다.

Page Fault Control Flow

PTE는 disk address에 대한 page의 PFN과 같은 데이터를 위해 사용된다.

OS가 page fault를 수신할 때, OS는 PTE를 살펴보고 disk에 요청을 보낸다.



Reference -> Trap -> Page가 존재하는지 Storage 확인 -> Page 가져옴 -> Page Table 리셋 -> 명령어 재실행

OS 보안

OS 위에서 실행되는 모든 S/W

안전하지 않은 OS -> 모든 SW가 안전하지 않음
안전하지 않은 OS 위의 보안적으로 완벽한 SW

모래 위에 집 짓기와 같다.
-> SW를 쉽게 공격할 수 있음 (memory, CPU)

OS 위의 수많은 프로그램들

하나의 process는 다른 process에 영향을 주어서는 안 된다.
하나의 process는 다른 process에 의해 방해받아서 안 된다.

다른 어떤 프로그램도 신뢰하지 마라 (특히 다운로드된 프로그램).

나쁜 프로그램(Bad program)의 위험성

disk에 있는 모든 것을 지우기

다른 process를 중단시키기

다른 컴퓨터들에 스팸 메일 보내기

등등을 할 수 있다.

Security Goal

Confidentiality

기밀성. 다른 사람들이 당신의 secret data를 읽지 못하도록 허용하지 않는 것이다.

e.g. Credit card number, PIN, Password

Integrity

무결성. adversary가 어떤 정보나 system의 component를 바꾸지 못하도록 하는 것이다

(읽는 것은 허용하되, 바꾸지는 못하게 함).

integrity의 한 가지 측면은 **authenticity**이다: 정보가 바뀌지 않았을 뿐만 아니라, adversary가 아니라 **신뢰할 수 있는 사람(trusted person)**에 의해 생성되었다는 점이다.

Availability

가용성. 어떤 정보나 service가 당신 자신 또는 다른 사람들의 사용을 위해 항상 사용 가능한 상태로 유지되는 것이다.

공격자가 무언가를 사용을 방해하지 못하도록 확실히 차단하는 것이다 (e.g. DDoS를 방지하기)

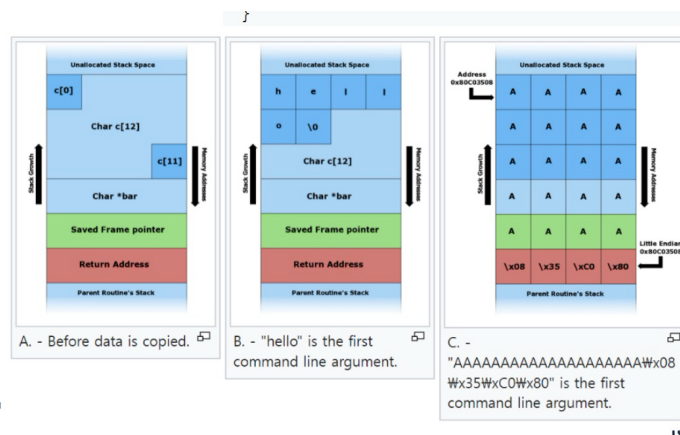
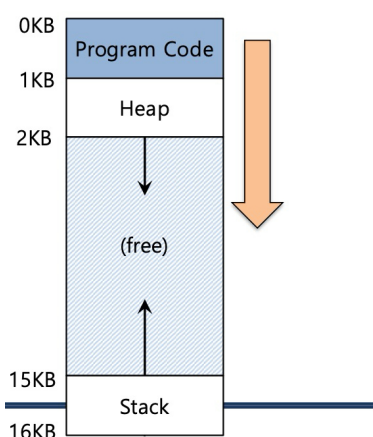
Stack Overflow

buffer overflow 취약점의 한 종류이다.

데이터 크기보다 더 큰 데이터를 쓰며, **결과적으로 return address를 덮어쓴다.** (return address 영역을 침범함)

hacker의 코드로 리다이렉트하도록 return address를 조작한다.

원래는 함수 실행이 끝나면 원래 자리로 돌아가야 하지만, return address가 다른 값으로 적혀있기 때문에 악성 코드를 실행하는 다른 해커의 코드로 넘어가게 된다!



```
#include <string.h>

void foo(char *bar)
{
    char c[12];

    strcpy(c, bar); // no bound. checking
}

int main(int argc, char **argv)
{
    foo(argv[1]);
    return 0;
}
```

Economy of mechanism

시스템은 최대한 다르고 심플하게 만드는 게 좋다.
심플하면 보고도 좋고 이해하기에도 편하기 때문이다.

Fail-safe defaults

불안전함이 아니라, security를 기본값으로 설정하는 것이다.
만약 선택 가능한 옵션이 있다면 덜 안전한 옵션보다 **더 안전한 옵션**을 선택해야 한다
e.g. 기본적으로 포트를 열어놓지 않는 방식

Complete mediation

: 행동이 취해질 때마다 수행될 행동이 security policies를 따르고 있는지를 매번 항상 체크해야 함

Open design

당신의 적이 당신의 design의 모든 상세 내용을 알고 있다고 가정하라.
만약 시스템이 어쨌든 그것의 security goals를 달성할 수 있다면, 당신은 좋은 상태에 있는 것이다.
해커가 그 시스템의 모든 부분을 알고 있다고 가정하고 보안을 설계해야 하며, 실제로 해커들은 모든 것을 다 알고 있다
-> 그래서 오픈소스가 더 안전할 수도 있다. 많은 사람들의 해킹 시도와 수정을 거쳤기 때문.
-> 클로즈 소스는 모든 약점이 검증되지 않은 상태이다

Separation of privilege

중요한 행동을 수행하기 위해 **분리된 parties 또는 credentials**를 요구하는 것이다.
중요한 일이 있을 때 한 명이 독단적으로 결정하는 게 아니라 권한을 나누어서 행사하는 게 좋다
e.g. 핵 미사일 버튼, 회사에서 결재

Least privilege

user 또는 process에게 당신이 허용하고자 하는 행동을 수행하는 데 요구되는 최소한의 privileges만 부여
권한을 많이 주는 게 아니라, 최소의 권한만 주어서 필요한 일만 할 수 있게 만드는 것이 좋다

Least common mechanism

서로 다른 users 또는 processes에 대해, 분리된 data structures 또는 mechanisms를 사용하는 것이다.
여러 컴포넌트가 공유 변수나 글로벌 변수 같은 것을 많이 공유하여 쓰면 시스템이 굉장히 위험해진다
-> 되도록 코드를 컴포넌트화해서 공유 변수를 줄여야 보안에 안전하다.

Acceptability

만약 당신의 users가 그것을 사용하지 않는다면, 당신의 system은 가치 없는 것이다.
아무리 보안적으로 안전한 프로그램이라도 너무 쓰기가 불편하면 사람들이 결국 안 쓰게 된다

TPM(Trusted Platform Module)

노트북 또는 데스크톱 컴퓨터에 있는 특수한 HW chip이며, 통합된 cryptographic keys로 hardware를 안전하게 보호하도록 설계된 것이다.

TPM은 **user의 신원을 증명**하는 것을 도우며 그들의 장치를 인증한다. 펌웨어가 변조되었는지 check!
ransomware attacks라든지 시스템 권한을 탈취하려는 **firmware 공격에 대한 실질적인 방어** 작용을 수행

Windows 11을 설치하기 위해서는 TPM 2.0 HW가 요구된다.

Identity Management

User

System users : 설치 동안 시스템에 의해 생성되며, system services와 applications를 실행하기 위해 사용됨.

(예시: root)

Regular users : 관리자에 의해 생성되며(created), 그들의 permissions에 기반하여 시스템과 그것의 resources에 접근할 수 있다 (예시: guest)

System Command

Sudo: 시스템 관리자(즉, root)로서 명령어를 실행하는 것

e.g. sudo useradd jaewoo, sudo userdel jaewoo

/etc/passwd: user information

유저 정보

/etc/shadow: encrypted password

암호화된 비밀번호

Group

users의 집합 e.g. Developers

'jaewoo'는 regular user이며 (홈 디렉토리: /home/jaewoo)

developers의 구성원이다 (/group/developers에 접근할 수 있다).

groups를 생성하고 관리하는 것은 특히 permissions를 다룰 때, 동시에 수많은 users를 다루는 가장 단순한 방법들 중 하나다.

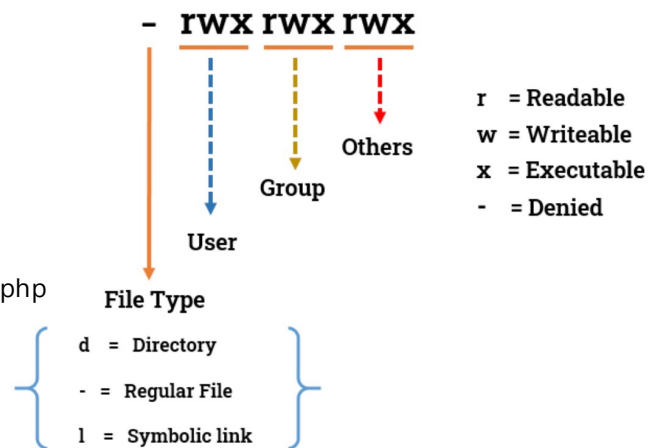
/etc/group

그룹 정보

리눅스에서 Access Control

e.g.

```
-rwxr--r-- 1 tom developers 139 Dec 13 07:07 activate.php
drwxrwxr-x 2 tom developers 4096 Dec 13 07:06 CSS
```



SELinux (Security-Enhanced Linux)

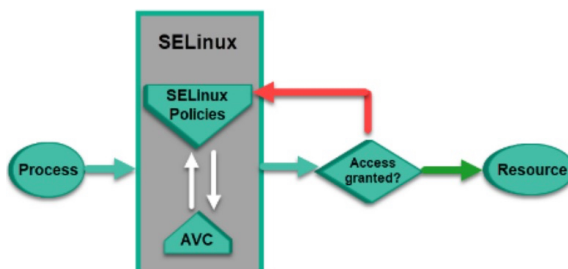
보안 강화 리눅스. 미국 NSA에서 개발하여 리눅스 환경의 보안 유출 자체를 방어할 수 있도록 만든 별도의 보안 모듈
주로 이 모듈은 Fedora와 RHEL(Red Hat Enterprise Linux) 배포판 환경에서 주로 탑재되어 사용

SELinux 보안 정책

기본적으로 모든 applications와 users를 차단하며, security policies에 지정된 것들에 대해서만 접근을 허용
만약 SELinux policy를 위반한다면 Root도 포트를 열 수 없다.

Access Vector Cache (AVC)는 SELinux policies에 대한 조회를 줄이기 위해 SELinux의 결정을 캐싱한다.

한 번 허용 여부가 결정되고 나면 이 결정을 AVC라는 공간에 저장하여 마치 캐시처럼 사용



10주차 실습 Virtual Memory

Virtual Memory가 왜 필요할까?

현대 시스템은 **동시에** 여러 프로세스를 실행합니다.

각 프로세스는 **자신만의 메모리 공간**을 필요로 합니다.

protection이 없다면, 한 프로세스가 **다른 프로세스의** 메모리에 접근하거나 이를 오염시킬 수 있습니다.

가상 메모리는 OS가 다음을 제공하도록 돕습니다:

Transparency(투명성) - 프로세스는 메모리 가상화가 일어나고 있는지 인지하지 못합니다.

Efficiency - 시간과 공간 측면에서 오버헤드(부가적인 비용)를 최소화

Protection - 각 프로세스를 서로서로 격리합니다.

Address Space

주소 공간은 **물리 메모리의 추상화**입니다.

실행 중인 프로세스의 메모리 영역들을 포함합니다.

프로세스 주소 공간은 보통 다음을 포함합니다:

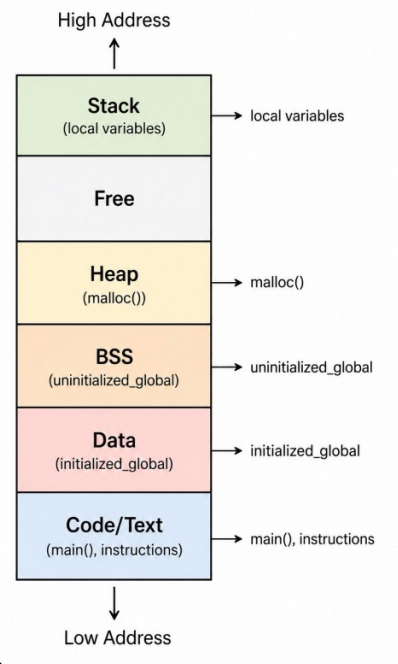
Code	코드 명령어.	main function
Data	전역변수(초기화O)	
BSS	전역변수(초기화X)	
Heap	동적할당	malloc()
Stack	지역변수, 매개변수, 리턴주소	main()의 지역 변수

모든 주소는 가상 주소이다

실행 중인 user program이 사용하는 모든 주소는 Virtual Address입니다.

e.g.

- Function address (함수 주소)
- Global variable address (전역 변수 주소)
- Local variable address (지역 변수 주소)
- Heap address returned by malloc() (malloc() 함수에 의해 반환되는 힙 주소)



실제 데이터는 **Physical Memory**에 저장됩니다.

OS와 하드웨어는 가상 주소를 물리 주소로 변환합니다.

Virtual Address Space 레이아웃

유저 프로세스는 메모리를 하나의 단순한 덩어리(블록)로 바라보지 않으며, 서로 다른 목적을 가진 **다양한 영역들**을 가지고 있습니다.

Low address → Code / Text → Data → BSS → Heap

High address → Stack

Heap과 Stack

Heap

동적 메모리 할당을 위해 사용됨

프로그램에 의해 메모리가 **명시적으로 요청**됩니다 - malloc()은 **힙**으로부터 메모리를 할당

Stack

함수 호출 중에 자동으로 사용됩니다.

지역 변수, 함수 인자(Arguments), 그리고 리턴 주소(Return addresses)를 저장합니다.

핵심 차이점 : **힙** 메모리는 **명시적으로** 요청되는 반면, **스택** 메모리는 **자동으로** 사용됩니다.

Memory API

애플리케이션은 raw physical memory를 직접 관리하지 않음.

이들은 다음과 같은 **메모리 API**들을 사용합니다: `malloc()`, `free()`, `calloc()`, `realloc()`

이 함수들은 **힙(heap) 영역의 메모리**를 관리하는 것을 돕습니다.

이번 실습에서 우리는 `malloc()`과 `free()`에 집중합니다.

malloc()

기능 : 힙 영역에 메모리 공간을 할당

```
void* malloc(size_t size)
```

매개변수 : 'size' - 바이트 단위로 요청된 메모리 블록의 크기

리턴 : 성공 시 - 할당된 메모리 블록을 가리키는 **포인터**

실패 시 - NULL 포인터

e.g. `int *arr = malloc(sizeof(int) * 5);`

의미 : 'size'는 프로그램이 요청한 **바이트 수**

힙 블록의 시작 주소를 반환합니다.

요청된 크기와 내부 힙 블록의 크기가 항상 일치하지 않을 수 있습니다.

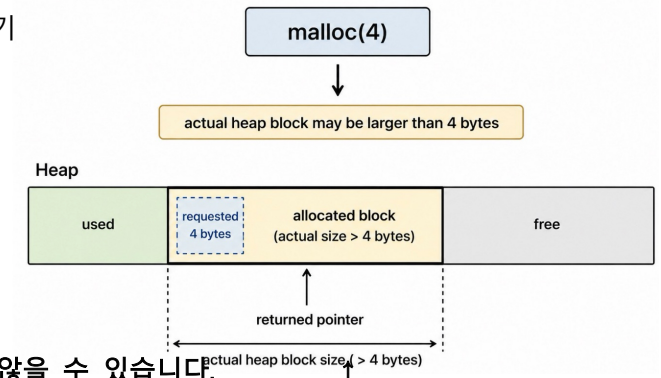
-> 왜일까요?

Allocator가 메타데이터(metadata)를 사용할 수 있습니다.

Allocator가 alignment(정렬)을 사용할 수 있습니다.

Allocator가 최소 블록 크기(minimum block size)를 사용할 수 있습니다.

*그래서 `malloc(4)`가 다음 힙 주소가 정확히 4바이트 떨어져 있음을 항상 의미하는 것은 아닙니다.



11주차 실습 Virtual Memory2

Address Translation

Program Address $\neq$ Physical Memory Address

Program 가상 주소를 사용한다.

자신의 주소 공간이 0부터 시작한다고 가정합니다.

Operating System + Hardware

가상 주소를 물리 주소로 변환합니다.

올바르지 않은 접근으로부터 메모리를 보호합니다.

Physical Memory

실제 데이터와 명령어를 저장

Virtual Address vs Physical Address

Virtual Address (VA)

프로그램에 의해 사용되는 주소

각 프로세스는 자신만의 가상 주소 공간을 가짐

프로세스 관점(view)에서는 0부터 시작

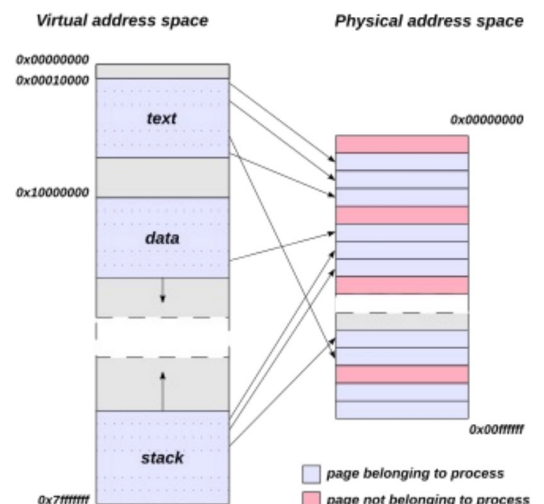
Physical Address (PA)

물리 메모리 내의 실제 주소입니다.

하드웨어가 RAM에 접근하기 위해 사용합니다.

Address Translation

VA -> PA



Address Translation

주소 변환은 가상 주소를 물리 주소로 변환합니다.

왜?

프로그램들은 격리된 주소 공간을 필요로 합니다.

여러 프로세스가 물리 메모리를 공유합니다.

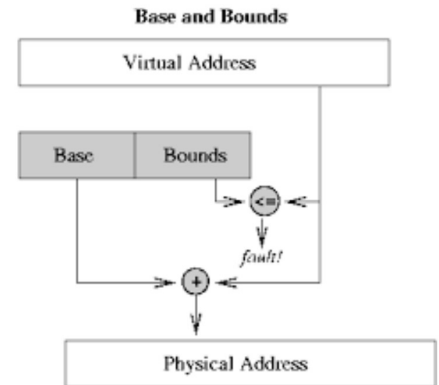
Invalid한 메모리 접근은 반드시 차단되어야 합니다.

핵심 아이디어

프로그램이 가상 주소를 생성합니다.

하드웨어가 이를 물리 주소로 변환합니다.

OS가 변환 정보를 설정합니다.



Base and Bound Translation

Base Register : 프로세스의 시작 물리 주소

Bound Register : 프로세스 주소 공간의 크기 제한 범위

Transition Rule : $PA = VA + base$

Valid Address Condition : $0 \leq VA < bound$

Base and Bound 예제

★★출제★★

base = 32KB = 32768 (32*1024)

bound = 16KB = 16384 (16*1024) 라 하자.

ex1) VA = 100

PA = 32768 + 100 = **32868**

ex2) VA = 4200

PA = 32768 + 4200 = **36968**

Limitation of Base and Bound

Base-and-Bound 방식은 전체 주소 공간을 하나의 연속적인 영역(contiguous region)으로 취급

Address Space

Code (코드) - Heap (힙) - Free Space (자유 공간) - Stack (스택)

↓

문제점 : 프로세스 영역 안에 사용되지 않는 free space가 여전히 물리 메모리를 차지할 수 있다.

메모리에 거대한 연속적 주소 공간을 배치하기 어려움.

Segmentation

세그멘테이션은 주소 공간을 논리적인 단위(logical parts)들로 나눕니다.

Common Segments

Code Segment: 명령어가 포함되어 있는 영역

Heap Segment: 동적 할당 메모리를 요청할 때 사용하는 영역

Stack Segment: 지역 변수와 함수의 호출 정보가 저장되어 있는 영역

각각의 Segment는 다음을 가진다

1. Virtual start address
2. Physical base address
3. Segment size

Segment Translation

Translation Steps :

1. VA가 어느 세그먼트에 포함되어 있는지 찾습니다.
2. 오프셋을 계산합니다 오프셋 = VA - Segment Start Address
3. PA를 계산합니다 PA = Segmentation base Address + Offset

Segment Translation 예제

HEAP Segment

Virtual start address = 4096

Physical base address = 34KB = 34816

VA = 4200 이라 할 때

offset = 4200 - 4096 = 104

PA = 34816 + 104 = 34920

Page Table Translation

페이징은 메모리를 고정된 크기의 블록으로 나눕니다.

Virtual Memory - 페이지(pages) 단위로 나눕니다.

Physical Memory - 프레임(frames) 단위로 나눕니다.

Page Table - 가상 페이지들을 물리 프레임들에 매핑합니다.

VPN -> PFN

Page Table 예제

Given : PAGE_SIZE = 4096

VA = 4200

Steps

VPN = 4200 / 4096 = 1

offset = 4200 % 4096 = 104

page_table[1] = 7 // 1번 페이지가 Physical 7번 프레임에 지정되었다!

Result PA = (7 x 4096) + 104 = 28672 + 104 = 28776

Hardware Page Table Translation

Translation Flow

CPU가 가상 주소를 생성합니다.

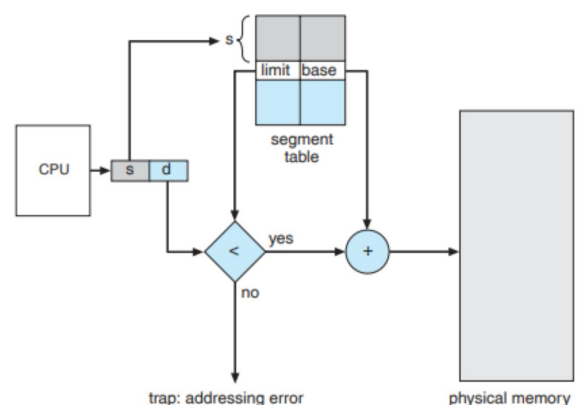
MMU(하드웨어 장치)가 VA를 VPN과 오프셋으로 나눕니다.

MMU가 페이지 테이블을 읽습니다.

MMU가 물리 프레임 번호를 찾습니다

MMU가 물리 주소를 계산합니다.

공식 : PA = frame * PAGE_SIZE + offset



12주차 실습 Memory API, Memory Allocate

Memory API란?

프로그램은 실행하는 동안 메모리가 필요하다.

Program

변수, 배열, 문자열, 포인터를 사용합니다.

temporary 데이터를 위한 메모리가 필요합니다.

실행 중에 메모리를 요청할 수 있습니다.

Operating System

각 프로세스에게 메모리를 제공합니다.
프로세스 Address Space을 관리합니다.
프로세스가 종료될 때 메모리를 회수합니다.

Memory API

프로그램이 힙 메모리를 요청하고 해제할 수 있도록 허용
주요 함수: malloc(), free()

Stack vs Heap

Stack 지역 변수와 함수 호출을 위해 사용된다.
 자동으로 관리된다.
 함수 호출과 리턴(반환)에 의해 크기가 늘어나고 줄어듭니다.

Heap 동적으로 할당된 메모리를 위해 사용됩니다.
 프로그래머(개발자)에 의해 관리됩니다.
 malloc()을 사용해 할당되고, free()를 사용해 해제됩니다.

Stack memory is automatic, heap memory must be managed carefully

스택 메모리는 자동인 반면, 힙 메모리는 신중하게 관리되어야 합니다.

포인터 변수

포인터는 변수를 저장한다

```
int num = 10;                                  int *ptr;                                  ptr = &num;
```

num은 값 10을 저장합니다.

ptr은 num의 주소를 저장합니다.

*ptr은 해당 주소에 있는 값에 접근합니다.

&num은 num의 주소를 의미합니다

ptr → address, *ptr → value

포인터 표현

선언 : int *pi;

pi가 정수형 배열을 가리킨다면?

pi[0] == *(pi + 0) pi[1] == *(pi + 1) pi[4] == *(pi + 4)

Important

pi + 1은 다음 정수(int) 위치로 이동합니다 1바이트가 더해지는 것 아님! 4바이트 더해짐
만약 int가 4바이트라면, pi + 1은 앞으로 4바이트 이동합니다.

혼동 주의

*pi + 4와 *(pi + 4)는 다르다! pi[0] = 10; pi[1] = 20; pi[4] = 50; 라 하면

*pi + 4 // 10 + 4 = 14

*(pi+4) // 50

괄호를 잘 보자!

malloc()

malloc()은 힙(heap) 영역에 메모리를 할당한다.

```
int *pi; pi = (int *)malloc(sizeof(int) * 5);
```

정수 5개를 위한 공간을 할당합니다.

힙 블록의 시작 주소를 반환하고, 이를 pi에 저장합니다.

```
if (pi == 0) { printf("malloc failed\n"); exit(1); } //항상 malloc 실패 여부를 확인해야 한다!
```

String과 Heap Buffer

문자열은 문자들을 위한 공간과 '\0' 널문자를 위한 공간이 필요합니다.

```
char *src = "hello";
```

Memory contents: h e l l o \0

strlen("hello")는 5를 반환하지만, 실제 필요한 메모리는 6바이트

```
dst = malloc(strlen(src) + 1); //NULL을 위해서 1바이트 추가
```

free()

free()는 malloc()에 의해 할당된 힙 메모리를 해제합니다.

```
int *p; p = malloc(sizeof(int) * 4); free(p);
```

free(p)는 힙 블록을 해제합니다.

free(p)는 포인터 변수 p를 지우지 않습니다.

p는 여전히 이전 주소를 포함하고 있을 수 있습니다.

-> 그래서 p = 0;을 해서 유효하지 않은 메모리를 실수로 사용하는 것을 방지합니다.

Dangling Pointer 땡글링 포인터는 다음과 같은 상황에서 발생합니다:

조건 메모리가 이미 해제되었습니다.

하지만 포인터가 여전히 이전 주소를 가지고 있습니다.

```
int *p = malloc(sizeof(int) * 4); free(p);
```

free(p) 호출 이후: 힙 블록은 유효하지 않지만, p는 여전히 이전 주소를 가지고 있습니다.

-> 유효하지 않은 메모리 접근, 예상치 못한 값, 데이터 손상을 일으킬 수 있다.

Memory Leak 메모리가 할당되었으나, 계속 해제되지 않습니다.

```
char *p = malloc(128); // free(p) is missing
```

프로그램이 힙 메모리를 계속해서 사용합니다.

결국 메모리가 고갈될 수 있습니다.

오랫동안 실행되는(long-running) 프로그램이 특히 심각한 영향을 받습니다.

-> Memory leak는 할당된 메모리가 시스템에 반환되지 않았음을 의미합니다.

13주차 실습 Paging, TLB, Swapping

Page Table Entry

하나의 Page Table Entry, 즉 PTE는 하나의 Virtual page에 대한 정보를 저장한다.

PTE에 있는 정보

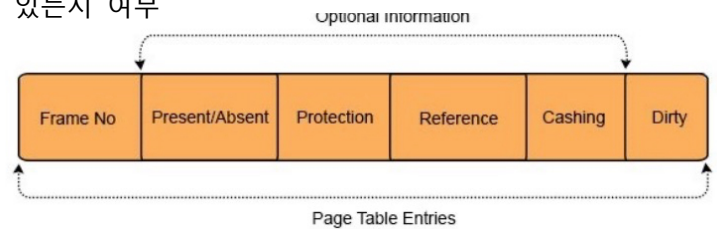
Physical Frame Number : Physical memory에서 page의 위치

Valid bit : 이 PTE가 유효한지 여부

Present bit : Page가 현재 physical memory에 존재하는지 여부

Writable bit : Write access가 허용되는지 여부

User bit : User mode에서 이 page에 접근할 수 있는지 여부



Page Table과 Protection

한 page table은 translation과 protection 모두를 위해 사용된다.

Translation : Virtual page를 physical frame에 매핑한다.

Protection : 유효하지 않거나 안전하지 않은 memory access를 차단한다.

안전하지 않은 Access의 예시

유효하지 않은 page에 access하는 것.

Read-only page에 write하는 것.

User program이 관리자나 커널만 접근할수 있는 page에 access하는 것

현재 memory에 존재하지 않는 page에 access하는 것

나올 수 있는 결과

ALLOW

DENY

PAGE FAULT

PTE Access Decision

Memory access는 PTE flag를 사용하여 검사된다.

만약 valid bit가 0이면 -> DENY

만약 present bit가 0이면 -> PAGE FAULT

만약 user mode에서 supervisor-only page에 접근하면 -> DENY

만약 write access가 read-only page를 대상으로 하면 -> DENY

만약 모든 검사를 통과하면 -> ALLOW

Why TLB?

모든 memory access는 address translation을 필요로 한다.

TLB가 없다면,

CPU가 virtual address를 생성한다. -> MMU가 page table을 읽는다. -> MMU가 physical frame을 찾는다.

-> MMU가 physical address를 계산한다.

문제점 : Page table은 memory에 저장된다.

그러므로, page table lookup은 추가적인 memory 접근 오버헤드를 더한다.

TLB : Translation Lookaside Buffer

TLB는 MMU 내부의 작은 하드웨어 캐시(cache)다

최근에 사용된 주소 변환(address translation) 정보를 저장한다.

한 TLB entry는 다음을 포함한다.

Virtual Page Number

Physical Frame Number

Permission information

★MMU는 page table을 읽기 전에 TLB를 먼저 확인한다.

TLB Hit

찾고자 하는 VPN이 이미 TLB 안에 있는 경우

-> Translation이 재사용됨, Page Table Lookup이 필요하지 않음.

TLB Miss

VPN이 TLB에 저장되어 있지 않다.

-> **Page Table**이 확인되어야 한다, **VPN**이 **TLB**에 삽입될 수 있다.

Locality 지역성

Temporal Locality : **최근에** access된 memory는 곧 다시 access될 수 있다.

Spatial Locality : **근처의** memory address들은 곧 다시 access될 수 있다.

e.g. VA 0, 4, 8, 12, 16

이 virtual address들은 서로 다르다.

하지만, PAGE_SIZE = 4096(바이트)이면, 그것들은 **같은 page**에 속한다.

그것들은 같은 VPN을 가진다.

첫 번째 MISS 이후에, 나중의 access들은 **HIT**이 될 수 있다.

Physical memory는 제한적이다

한 process는 많은 virtual page를 가질 수 있다.

Physical memory는 **제한적인 수의 frame**을 가진다.

모든 virtual page가 동시에 physical memory에 머무를 수는 없다.

OS는 **일시적으로 page들을 disk로 이동**시킬 수 있다.

Swap Space : Page들을 일시적으로 저장하기 위해 사용되는 **disk 영역**

Swap Out : 한 page를 physical memory에서 swap space로 이동시키는 것

Swap In : 한 page를 swap space에서 physical memory로 다시 가져오는 것

Present Bit and Page Fault

Present bit는 한 page가 physical memory에 있는지 알려준다.



present = 1 Page가 현재 physical memory에 있다. Access가 계속될 수 있다.

present = 0 Page가 현재 physical memory에 없다. Page가 **swap space**에 있을 수 있다.
Access는 **PAGE FAULT**를 발생시킨다.

만약 page가 valid하지만 현재 memory에 없다면 **page fault**는 **OS**에 의해 처리될 수 있다.

왜 Address Translation인가?

Program Address $\neq$ Physical Memory Address

Program

Virtual address를 사용한다.

실제 data와 instruction을 저장한다.

OS + Hardware

Virtual address를 physical address로 **translation**한다

Invalid access로부터 memory를 보호한다. 커널의 메모리를 침범하지 않도록 안전하게 보호

Physical Memory

실제 data와 instruction을 저장한다.

Page Replacement

Page fault가 발생하면, 요청된 page는 memory로 load되어야 한다.

만약 비어 있는 frame이 존재한다면:

요청된 page를 비어 있는 frame에 load한다

만약 physical memory가 가득 차 있다면:

Victim page를 선택한다.

Victim page를 swap out한다.

요청된 page를 swap in한다.

page replacement policy : 어떤 page가 제거되어야 하는지 결정

Replacement policy들은 page 사용 정보를 사용할 수 있다.

e.g. Accessed bit

최근에 사용된 page들은 보호되어야 한다.

최근에 사용되지 않은 page들은 victim이 될 수 있다.

LRU Page Replacement

Least Recently Used 오랫동안 사용되지 않은 것 내보내기

최근에 사용된 page들은 곧 다시 사용될 수 있다.

오랜 시간 동안 사용되지 않은 page들이 더 좋은 victim 후보들이다.

e.g. Frames: [1 2 3]

최근 access 순서: 1 $\rightarrow$ 3 $\rightarrow$ 1

Page 2는 최근에 사용되지 않았다. $\rightarrow$ Victim Page : 2

문제점 : 정확한 LRU는 access history를 추적하는 것을 요구한다.

이것은 실제 시스템에서 비용이 많이 들 수 있다

Second-Chance Page Replacement

accessed bit를 사용하여 LRU와 비슷하게 구현.

accessed = 1은 그 page가 최근에 사용됨

accessed = 0은 그 page가 최근에 사용되지 않았음

만약 accessed = 1이면

그 page에 세컨드 찬스(두 번째 기회)를 준다.

Accessed bit를 0으로 만들고 $\rightarrow$ 다음 frame으로 이동

만약 accessed = 0이면

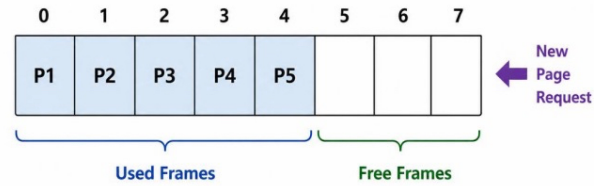
victim page로 선택

$\rightarrow$ LRU는 전체 히스토리를 추적하지만, Second Chance는 비트 하나로 해결

14,15주차 실습 Page Daemon and Watermarks + OS Security

Page Daemon and Watermarks

low watermark와 high watermark를 사용하여 memory reclamation을 시뮬레이션하기
free frames의 개수를 감시하기
free memory가 낮을 때 memory reclamation을 시작하기
충분한 frames가 회수된 후에 memory reclamation을 중지하기



물리 메모리 크기는 제한적이다

오직 **제한된 수의 Pages**만 동시에 Memory에 머무를 수 있다.
Processes는 계속해서 Pages를 요청한다. 각각의 새로운 Page allocation은 하나의 **Free frame**을 사용한다.
Free Frames는 점진적으로 감소한다.
Operating System은 미래의 요청을 위해 Frames를 준비해야 한다.

Lazy Memory Reclamation

Memory 회수를 최대한 늦게 수행하는 방식을 의미한다.
Lazy approach는 Free frame이 여전히 존재하는 동안에는 Memory를 회수하지 않는다.
Reclamation이 너무 늦게 시작된다
OS는 다른 Frame이 긴급하게 필요할 때만 Memory 회수를 시작한다.
요청하는 Process는 기다려야 한다
Frame이 회수될 때까지 Memory allocation은 계속될 수 없다.

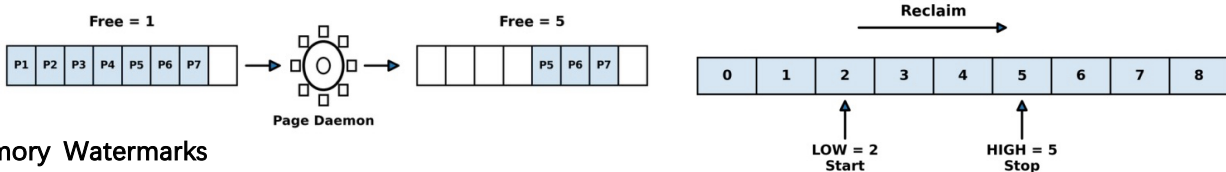
-> 개선책 : Free-frame 개수가 0에 도달하기 전에 Free frames를 준비하기

Page Daemon

Daemon : 특정 시스템 조건이 발생하기를 기다리는 Background task
Page Daemon : 가용할 수 있는 **Physical memory frames**의 개수를 감시한다
Memory Reclamation : Free memory가 너무 낮아지면 차지된 **Frames**를 **회수**한다

충분한 Free frames가 확보되면 다시 대기 상태로 돌아간다.

Wait → Low Free Memory → Reclaim Frames → Enough Free Memory → Wait



Memory Watermarks

Low Watermark

Memory reclamation이 언제 시작할지 결정한다. 시작 임계값(Threshold)
Free-frame count가 **LOW**보다 작아질 때 Page daemon이 시작한다.
 $free < LOW \rightarrow$ Page Daemon 시작

High Watermark

Memory reclamation이 언제 멈출지 결정한다.
Free-frame count가 **HIGH**에 도달할 때까지 Page daemon이 계속된다.
 $free == HIGH \rightarrow$ Page Daemon 정지 Stop Target이다. 정지할 때를 알려줌

LOW와 HIGH 사이의 간격은 시작과 중지 행위를 방지하기 위해서 만들어졌다. 1개만 있었다면 계속 중지됨.
Operating system은 가용한 Frames의 안정적인 예비분을 유지한다.

제한된 Process

보안 상태 추가

각 Process가 normal인지 또는 restricted인지를 저장한다.

Restricted 모드로 집입

setrestricted() system call을 사용한다.

위험한 Operations를 차단

파일 삭제를 차단하기

Process 종료를 차단하기

왜 Process 보호해야 하나?

여러 개의 Processes가 OS 상에서 실행된다.

Processes는 CPU, memory, 파일들, 그리고 다른 system resources를 공유한다.

하나의 Process가 다른 것에 영향을 주어서는 안 된다.

하나의 Process는 다른 Process를 방해해서는 안 된다.

악성 프로그램들은 **System에 손상**을 줄 수 있다.

중요한 파일들을 삭제한다.

다른 Processes를 종료시킨다.

System resources를 오용(misuse)한다.

Kernel 보호

Kernel은 중요한 System operations에 대한 접근을 제어한다.

File Permissions

▪ r

- Allows users to view the contents of a file

▪ w

- Allows users to modify or overwrite a file

▪ x

- Allows users to run a file as a program

▪ Permission Groups

- Owner
- Group
- Others

▪ Change File Permissions

- chmod changes the read, write, and execute permissions of a file

▪ Numeric Values

- Read: 4
- Write: 2
- Execute: 1

▪ Permission Calculation

- $7 = 4 + 2 + 1 = \text{rwx}$
- $6 = 4 + 2 = \text{rw-}$
- $5 = 4 + 1 = \text{r-x}$
- $4 = 4 = \text{r--}$

```
[(base) obyungmoon@obyeeonghun-ai-MacBookPro xv6-labs-2025 % cat > chmod.txt
safe
[(base) obyungmoon@obyeeonghun-ai-MacBookPro xv6-labs-2025 % cat chmod.txt
safe
[(base) obyungmoon@obyeeonghun-ai-MacBookPro xv6-labs-2025 % ls -l chmod.txt
-rw-r--r--  1 obyungmoon  staff  5  6 15 16:00 chmod.txt
[(base) obyungmoon@obyeeonghun-ai-MacBookPro xv6-labs-2025 % chmod 444 chmod.txt
[(base) obyungmoon@obyeeonghun-ai-MacBookPro xv6-labs-2025 % ls -l chmod.txt
-r--r--r--  1 obyungmoon  staff  5  6 15 16:00 chmod.txt
```

▪ cat > chmod.txt

▪ cat chmod.txt

▪ ls -l chmod.txt

▪ chmod 444 chmod.txt

▪ ls -l chmod.txt

vaddr.c

```
int initialized_global[1024] = {1}; // Data area
int uninitialized_global[1024]; // BSS area

int
main(int argc, char *argv[])
{
    int local1;
    int local2;

    int *heap1 = malloc(sizeof(int));
    int *heap2 = malloc(sizeof(int));

    printf("=== xv6 virtual address space ===\n");

    printf("code main          : %p\n", main);
    printf("data initialized_global : %p\n", initialized_global);
    printf("data uninitialized_global : %p\n", uninitialized_global);
    printf("stack local1 : %p\n", &local1);
    printf("stack local2 : %p\n", &local2);
    printf("heap heap1 : %p\n", heap1);
    printf("heap heap2 : %p\n", heap2);

    return 0;
}
```

```
=== xv6 virtual address space ===
code main          : 0x4005d0
data initialized_global : 0x601040
data uninitialized_global : 0x602040
stack local1 : 0x7ffd5b3a1a0c
stack local2 : 0x7ffd5b3a1a08
heap heap1 : 0x602050
heap heap2 : 0x602060
```

malloc.c

```
int size = 16; //크기 바꿔보기!

printf("=== heap allocation and reuse experiment ===\n");
printf("malloc size = %d bytes\n", size);

char *a = malloc(size);
char *b = malloc(size);
char *c = malloc(size);

printf("\n[Step 1] malloc three blocks\n");
printf("a = 0x%lx\n", (uint64)a);
printf("b = 0x%lx\n", (uint64)b);
printf("c = 0x%lx\n", (uint64)c);

printf("\n[Step 2] add more malloc\n");

char *d = malloc(size);
char *e = malloc(size);
printf("d = 0x%lx\n", (uint64)d);
printf("e = 0x%lx\n", (uint64)e);

printf("\n[Step 3] free and malloc again\n");

free(a);
printf("a = 0x%lx\n", (uint64)a);
a = malloc(size);
printf("a = 0x%lx\n", (uint64)a);

// free and malloc again

exit(0);
```

```
=== heap allocation and reuse experiment ===
malloc size = 16 bytes

[Step 1] malloc three blocks
a = 0x1000
b = 0x1010
c = 0x1020

[Step 2] add more malloc
d = 0x1030
e = 0x1040

[Step 3] free and malloc again
a = 0x1000
a = 0x1000
```

base_bound.c

```
#define KB 1024
translate_base_bound(int va, int base, int bound)
{
    // TODO 1:
    // va가 0보다 작거나 bound 이상이면 -1을 반환하시오.
    if(va < 0 || va >= bound)
        return -1;

    // TODO 2:
    // 정상 범위이면 base + va를 반환하시오.
    return base + va;

    //return pa;
}

void
print_result(int va, int base, int bound)
{
    int pa = translate_base_bound(va, base, bound);

    if(pa < 0)
        printf("VA %d -> SEGMENTATION FAULT\n", va);
    else
        printf("VA %d -> PA %d\n", va, pa);
}

int
main(int argc, char *argv[])
{
    int base = 32 * KB;
    int bound = 16 * KB;

    printf("=== Base and Bound Translation ===\n");
    printf("base = %d, bound = %d\n", base, bound);

    print_result(100, base, bound);
    print_result(4200, base, bound);
    print_result(17000, base, bound);

    exit(0);
}
```

=== Base and Bound Translation ===
base = 32768, bound = 16384
VA 100 -> PA 32868
VA 4200 -> PA 36968
VA 17000 -> SEGMENTATION FAULT

segmentation.c

```
#define KB 1024
#define SEG_COUNT 3

struct segment {
    char *name;
    int va_start;
    int base;
    int size;
};

struct segment seg_table[SEG_COUNT] = {
    {"CODE", 0 * KB, 32 * KB, 2 * KB},
    {"HEAP", 4 * KB, 34 * KB, 2 * KB},
    {"STACK", 14 * KB, 28 * KB, 2 * KB}
};

int
translate_segmentation(int va)
```



```

{
    for(int i = 0; i < SEG_COUNT; i++){
        int start = seg_table[i].va_start;
        int end = seg_table[i].va_start + seg_table[i].size;

        // TODO 1:
        // va가 특정 segment 범위 안에 있는지 확인하시오.
        // 조건: va_start <= va < va_start + size
        if(va >= start && va < end){
            printf("VA %d is in %s segment\n", va, seg_table[i].name);

            // TODO 2:
            // offset = va - va_start 를 계산하시오.
            int offset = va-start;

            // TODO 3:
            // pa = segment base + offset 을 계산
            int pa = seg_table[i].base + offset;

            return pa;
        }

        return -1;
    }

    void
    print_result(int va)
    {
        int pa = translate_segmentation(va);

        if(pa < 0)
            printf("VA %d -> SEGMENTATION FAULT\n", va);
        else
            printf("VA %d -> PA %d\n", va, pa);
    }

    int
    main(int argc, char *argv[])
    {
        printf("=== Segmentation Translation ===\n");

        print_result(100);
        print_result(3000);
        print_result(15000);
        print_result(5000);

        exit(0);
    }
}

```

```

=== Segmentation Translation ===
VA 100 is in CODE segment
VA 100 -> PA 32868
VA 3000 -> SEGMENTATION FAULT
VA 15000 is in STACK segment
VA 15000 -> PA 29336
VA 5000 is in HEAP segment
VA 5000 -> PA 35720

```

page_table.c

```

#define PAGE_SIZE 4096
#define PAGE_COUNT 3

int page_table[PAGE_COUNT] = {
    3, // virtual page 0 -> physical frame 3
    7, // virtual page 1 -> physical frame 7
    5  // virtual page 2 -> physical frame 5
};

int
translate_page_table(int va)
{
    // TODO 1:

```

```

// vpn = va / PAGE_SIZE 계산
int vpn = va / PAGE_SIZE;

// TODO 2:
// offset = va % PAGE_SIZE 계산
int offset = va % PAGE_SIZE;

// TODO 3:
// vpn이 page table 범위를 벗어나면 -1 반환
if(vpn < 0 || vpn >= PAGE_COUNT)
    return -1;
// TODO 4:
// frame = page_table[vpn]
int frame = page_table[vpn];

// TODO 5:
// pa = frame * PAGE_SIZE + offset 계산
int pa = frame * PAGE_SIZE + offset;

return pa;
}

void
print_result(int va)
{
    int pa = translate_page_table(va);

    if(pa < 0)
        printf("VA %d -> PAGE FAULT\n", va);
    else
        printf("VA %d -> PA %d\n", va, pa);
}

int
main(int argc, char *argv[])
{
    printf("=== Page Table Translation ===\n");

    print_result(100);
    print_result(4200);
    print_result(9000);
    print_result(20000);

    exit(0);
}

```

=== Page Table Translation ===

VA 100 -> PA 12388

VA 4200 -> PA 28776

VA 9000 -> PA 21288

VA 20000 -> PAGE FAULT

ptr_expr.c

```

int
main(int argc, char *argv[])
{
    int *pi;

    //크기 5인 메모리 할당
    pi = (int *)malloc(sizeof(int) * 5);

    if(pi == 0){
        printf("malloc failed\n");
        exit(1);
    }

    pi[0] = 10;
    pi[1] = 20;

```

```

pi[2] = 30;
pi[3] = 40;
pi[4] = 50;

printf("=== Pointer Expression Practice ===\n");

printf("pi[1] = %d\n", pi[1]);
printf("(pi + 1) = %d\n", *(pi + 1));
printf("*pi + 4 = %d\n", *pi + 4);
printf("(pi + 4) = %d\n", *(pi + 4));

printf("=== Address Check ===\n");

printf("&pi[0] = %p\n", &pi[0]);
printf("&pi[1] = %p\n", &pi[1]);
printf("&pi[4] = %p\n", &pi[4]);

// 에러 출력
printf("=== Out of Bounds Access ===\n");

printf("(pi + 5) = %d\n", *(pi + 5));

free(pi);
exit(0);
}

```

```

=== Pointer Expression Practice ===
pi[1] = 20
*(pi + 1) = 20
*pi + 4 = 14
*(pi + 4) = 50
=== Address Check ===
&pi[0] = 0XXXXXXXXX
&pi[1] = 0XXXXXXXXX
&pi[4] = 0XXXXXXXXX
=== Out of Bounds Access ===
*(pi + 5) = YYYYYYYY

```

heapstr.c

```

int
main(int argc, char *argv[])
{
    char *src;
    char *dst;
    int len;
    int i;

    if(argc != 2){
        printf("usage: heapstr [message]\n");
        exit(1);
    }

    src = argv[1];
    len = strlen(src);

    //문자열 길이 + 1 만큼 메모리 할당
    dst = (char *)malloc(sizeof(char) * (len + 1));

    if(dst == 0){
        printf("malloc failed\n");
        exit(1);
    }

    // src 문자열을 dst로 복사 (\0 포함)
    for(i = 0; i <= len; i++){
        dst[i] = src[i];
    }

    printf("=== Heap String Copy ===\n");
    printf("src = %s\n", src);
    printf("dst = %s\n", dst);

    printf("\n=== Address Check ===\n");
    printf("src address = %p\n", src);
    printf("dst address = %p\n", dst);

```

```

=== Heap String Copy ===
src = hello
dst = hello

=== Address Check ===
src address = 0x7ffd5b3a1000
dst address = 0x602020

=== Length Check ===
strlen(src) = 5
allocated size = strlen(src) + 1 = 6

```

```

printf("\n=== Length Check ===\n");
printf("strlen(src) = %d\n", len);
printf("allocated size = strlen(src) + 1 = %d\n", len + 1);

free(dst);
exit(0);
}

```

pte.c

```

#define READ 0
#define WRITE 1
#define ALLOW 0
#define DENY 1
#define PAGE_FAULT 2
struct pte {
    int valid;
    int present;
    int writable;
    int user;
    int pfn;
};
char*
result_name(int result)
{
    if(result == ALLOW)
        return "ALLOW";
    else if(result == DENY)
        return "DENY";
    else if(result == PAGE_FAULT)
        return "PAGE FAULT";
    else
        return "UNKNOWN";
}
int
check_access(struct pte p, int access_type, int is_user)
{
    // TODO 1:
    // If the valid bit is 0, return DENY.
    if(p.valid == 0)
        return DENY;
    // TODO 2:
    // If the present bit is 0, return PAGE_FAULT.
    if(p.present == 0)
        return PAGE_FAULT;
    // TODO 3:
    // If a user process accesses a non-user page, return DENY.
    if(is_user && p.user == 0)
        return DENY;
    // TODO 4:
    // If the access type is WRITE and the page is not writable, return DENY.
    if(access_type == WRITE && p.writable == 0){
        return DENY;
    }
    // TODO 5:
    // If all checks pass, return ALLOW.
    return ALLOW;
}
void
test_access(char *name, struct pte p, int access_type, int is_user)
{
    int result = check_access(p, access_type, is_user);
    printf("%s -& %s\n", name, result_name(result));
}
int

```

```

main(int argc, char *argv[])
{
    printf("=== PTE Permission Checker ===\n");
    // TODO 6:
    // Create a PTE that produces ALLOW.
    struct pte allow_page = {1, 1, 1, 1, 0};
    // TODO 7:
    // Create a read-only page that produces DENY for WRITE access.
    struct pte readonly_page = {1, 1, 0, 1, 0};
    // TODO 8:
    // Create a supervisor-only page that produces DENY for user access.
    struct pte supervisor_page = {1, 1, 1, 0, 0};
    // TODO 9:
    // Create an invalid page that produces DENY.
    struct pte invalid_page = {0, 1, 1, 1, 0};
    // TODO 10:
    // Create a not-present page that produces PAGE FAULT.
    struct pte not_present_page = {1, 0, 1, 1, 0};
    test_access("ALLOW case: READ by user", allow_page, READ, 1);
    test_access("ALLOW case: WRITE by user", allow_page, WRITE, 1);
    test_access("DENY case: write to read-only page", readonly_page, WRITE, 1);
    test_access("DENY case: user access to supervisor page", supervisor_page, READ, 1);
    test_access("DENY case: invalid page", invalid_page, READ, 1);
    test_access("PAGE FAULT case: not-present page", not_present_page, READ, 1);
    exit(0);
}

```

```

=== PTE Permission Checker ===
ALLOW case: READ by user -> ALLOW
ALLOW case: WRITE by user -> ALLOW
DENY case: write to read-only page -> DENY
DENY case: user access to supervisor page -> DENY
DENY case: invalid page -> DENY
PAGE FAULT case: not-present page -> PAGE FAULT

```

tlb.c

```

#define PAGE_SIZE 4096
#define TLB_SIZE 2
int tlb[TLB_SIZE];
int next_replace = 0;
void
init_tlb(void)
{
    for(int i = 0; i < TLB_SIZE; i++){
        tlb[i] = -1;
    }
    next_replace = 0;
}
void
print_tlb(void)
{
    printf("TLB [");
    for(int i = 0; i < TLB_SIZE; i++){
        if(tlb[i] == -1)
            printf(" -");
        else
            printf(" %d", tlb[i]);
    }
    printf(" ]\n");
}
int
tlb_lookup(int vpn)
{
    // TODO 1:
    // Return 1 if vpn exists in the TLB.
    for(int i = 0; i < TLB_SIZE; i++){
        if(tlb[i] == vpn)
            return 1;
    }
    return 0;
}
void

```

```

tlb_insert(int vpn)
{
    // TODO 2:
    // Insert vpn into the TLB using FIFO replacement.
    // Store vpn at next_replace.
    // Move next_replace to the next position.
    tlb[next_replace] = vpn;
    next_replace = (next_replace + 1) % TLB_SIZE;
}

void
run_pattern(char *name, int pattern[], int n)
{
    init_tlb();
    printf("\n=== %s ===\n", name);
    for(int i = 0; i < n; i++){
        int va = pattern[i];
        // TODO 3:
        // Compute the VPN from the virtual address.
        int vpn = va / PAGE_SIZE;
        if(tlb_lookup(vpn)){
            printf("VA %d -& VPN %d -& HIT\n", va, vpn);
        } else {
            printf("VA %d -& VPN %d -& MISS\n", va, vpn);
            tlb_insert(vpn);
        }
    }
    print_tlb();
}

int
main(int argc, char *argv[])
{
    int pattern_a[] = {0, 4, 8, 12, 16};
    int pattern_b[] = {0, 4096, 8192, 12288, 0, 4096};
    int n_a = sizeof(pattern_a) / sizeof(pattern_a[0]);
    int n_b = sizeof(pattern_b) / sizeof(pattern_b[0]);
    printf("=== TLB Hit/Miss Simulator ===\n");
    run_pattern("Pattern A: Spatial Locality", pattern_a, n_a);
    run_pattern("Pattern B: Poor Locality", pattern_b, n_b);
    exit(0);
}

```

```
=== TLB Hit/Miss Simulator ===
```

```
=== Pattern A: Spatial Locality ===
```

```
VA 0 -> VPN 0 -> MISS
```

```
TLB [ 0 - ]
```

```
VA 4 -> VPN 0 -> HIT
```

```
TLB [ 0 - ]
```

```
VA 8 -> VPN 0 -> HIT
```

```
TLB [ 0 - ]
```

```
VA 12 -> VPN 0 -> HIT
```

```
TLB [ 0 - ]
```

```
VA 16 -> VPN 0 -> HIT
```

```
TLB [ 0 - ]
```

```
=== Pattern B: Poor Locality ===
```

```
VA 0 -> VPN 0 -> MISS
```

```
TLB [ 0 - ]
```

```
VA 4096 -> VPN 1 -> MISS
```

```
TLB [ 0 1 ]
```

```
VA 8192 -> VPN 2 -> MISS
```

```
TLB [ 2 1 ]
```

```
VA 12288 -> VPN 3 -> MISS
```

```
TLB [ 2 3 ]
```

```
VA 0 -> VPN 0 -> MISS
```

```
TLB [ 0 3 ]
```

```
VA 4096 -> VPN 1 -> MISS
```

```
TLB [ 0 1 ]
```

```
page_swap.c
```

```

#define PAGE_COUNT 6
#define FRAME_COUNT 3
struct page {
    int present;    // 1: in memory, 0: in swap
    int frame;      // physical frame number, -1 if not present
    int swap_block; // simulated swap block number
    int accessed;   // accessed bit
};
struct page page_table[PAGE_COUNT];
int frames[FRAME_COUNT];
int clock_hand = 0;
void
init_system(void)
{
    int i;
    for(i = 0; i < PAGE_COUNT; i++){
        page_table[i].present = 0;
        page_table[i].frame = -1;
        page_table[i].swap_block = i;
        page_table[i].accessed = 0;
    }
    for(i = 0; i < FRAME_COUNT; i++){
        frames[i] = -1;
    }
}

```



```

    clock_hand = 0;
}
void
print_frames(void)
{
    int i;
    printf("frames [");
    for(i = 0; i < FRAME_COUNT; i++){
        if(frames[i] == -1)
            printf(" -");
        else
            printf(" %d", frames[i]);
    }
    printf(" ]\n");
}
void
print_pte(int vpn)
{
    printf("PTE[%d]: present=%d, frame=%d, swap_block=%d, accessed=%d\n",
        vpn,
        page_table[vpn].present,
        page_table[vpn].frame,
        page_table[vpn].swap_block,
        page_table[vpn].accessed);
}
int
find_empty_frame(void)
{
    int i;
    for(i = 0; i < FRAME_COUNT; i++){
        if(frames[i] == -1)
            return i;
    }
    return -1;
}
void
handle_hit(int vpn)
{
    printf("HIT\n");
    // Recently accessed page.
    page_table[vpn].accessed = 1;
    print_pte(vpn);
}
void
swap_in_empty(int vpn, int frame)
{
    printf("swap in page %d from block %d\n",
        vpn, page_table[vpn].swap_block);
    frames[frame] = vpn;
    page_table[vpn].present = 1;
    page_table[vpn].frame = frame;
    page_table[vpn].accessed = 1;
}
void
swap_out_and_replace(int vpn, int victim)
{
    printf("memory is full\n");
    printf("victim page = %d\n", victim);
    printf("swap out page %d to block %d\n",
        victim, page_table[victim].swap_block);
    page_table[victim].present = 0;
    page_table[victim].frame = -1;
    page_table[victim].accessed = 0;
    printf("swap in page %d from block %d\n",
        vpn, page_table[vpn].swap_block);
    frames[clock_hand] = vpn;
    page_table[vpn].present = 1;
}

```

```

page_table[vpn].frame = clock_hand;
page_table[vpn].accessed = 1;
clock_hand = (clock_hand + 1) % FRAME_COUNT;
}
void
replace_second_chance(int vpn)
{
    int victim;
    while(1){
        victim = frames[clock_hand];
        printf("check page %d: accessed=%d\n",
            victim, page_table[victim].accessed);
        if(page_table[victim].accessed == 1){
            printf("second chance for page %d\n", victim);
            // TODO 1:
            // Clear the accessed bit.
            page_table[victim].accessed = 0;
            // TODO 2:
            // Move clock_hand to the next frame.
            clock_hand = (clock_hand + 1) % FRAME_COUNT;
        } else {
            printf("select victim page %d\n", victim);
            // TODO 3:
            // Swap out the victim page and swap in the new page.
            page_table[victim].present = 0;
            page_table[victim].frame = -1;
            page_table[victim].accessed = 0;
            break;
        }
    }
}
int
main(int argc, char *argv[])
{
    int refs[] = {1, 2, 3, 1, 4, 1, 2, 5};
    int n = sizeof(refs) / sizeof(refs[0]);
    int i;
    int vpn;
    int empty;
    init_system();
    printf("=== Page Fault and Second-Chance Replacement ===\n");
    for(i = 0; i < n; i++){
        vpn = refs[i];
        printf("\naccess VPN %d\n", vpn);
        if(page_table[vpn].present == 1){
            handle_hit(vpn);
        } else {
            printf("PAGE FAULT\n");
            printf("PTE[%d].present = 0\n", vpn);
            empty = find_empty_frame();
            if(empty &= 0){
                swap_in_empty(vpn, empty);
            } else {
                replace_second_chance(vpn);
            }
            print_pte(vpn);
        }
        print_frames();
    }
    exit(0);
}

```

=== TLB Hit/Miss Simulator ===

=== Pattern A: Spatial Locality ===

VA 0 -> VPN 0 -> MISS

TLB [0 -]

VA 4 -> VPN 0 -> HIT

TLB [0 -]

VA 8 -> VPN 0 -> HIT

TLB [0 -]

VA 12 -> VPN 0 -> HIT

TLB [0 -]

VA 16 -> VPN 0 -> HIT

TLB [0 -]

=== Pattern B: Poor Locality ===

VA 0 -> VPN 0 -> MISS

TLB [0 -]

VA 4096 -> VPN 1 -> MISS

TLB [0 1]

VA 8192 -> VPN 2 -> MISS

TLB [2 1]

VA 12288 -> VPN 3 -> MISS

TLB [2 3]

VA 0 -> VPN 0 -> MISS

TLB [0 3]

VA 4096 -> VPN 1 -> MISS

TLB [0 1]

page_daemon.c

```
#define TOTAL 8
#define LOW 2
#define HIGH 5
#define FREE -1
```

```
int frames[TOTAL] = {
    1, 2, 3, 4, 5, FREE, FREE, FREE
};
int requests[] = {
    6, 7, 8, 9
};
```

```
int
count_free_frames(void)                //빈 프레임 검사
{
```

```
    int count = 0;
```

```
    // TODO 1:
```

```
    // 모든 프레임을 검사한다.
```

```
    // frames[i] == FREE일 때 count를 증가시킨다.
```

```
    for (int i = 0; i < TOTAL; i++) {
```

```
        if (frames[i] == FREE) {
```

```
            count++;
```

```
        }
```

```
    }
```

```
    return count;
```

```
}
```

```
void
```

```
run_page_daemon(void)                // 빈 프레임의 개수가 HIGH에 도달할 때까지 차지된 프레임을 회수
```

```
{
```

```
    int evicted = 0;
```

```
    for (int i = 0; i < TOTAL; i++) {
```

```
        //차지된 프레임을 찾아 FREE로 변경한다.
```

```
        if (count_free_frames() >= HIGH) {
```

```
            // 빈 프레임 수가 HIGH에 도달하면 중지한다
```

```
            break;
```

```
        }
```

```
        if (frames[i] != FREE) {
```

```
            frames[i] = FREE;
```

```
            evicted++;
```

```
        }
```

```
    }
```

```
    printf("Daemon: evicted=%d, free=%d\n",
```

```
        evicted, count_free_frames());
```

```
}
```

```
int
```

```
main(int argc, char *argv[])
```

```
{
```

```
    int request_count =
```

```
        sizeof(requests) / sizeof(requests[0]);
```

```
    printf("LOW=%d HIGH=%d\n\n", LOW, HIGH);
```

```
    for(int r = 0; r < request_count; r++){
```

```
        int page = requests[r];
```

```
        for (int i = 0; i < TOTAL; i++) {
```

```
            // 하나의 FREE 프레임을 찾는다.
```

```
            if (frames[i] == FREE) {
```

```
                // 요청된 페이지를 해당 프레임에 저장한다.
```

```
                frames[i] = page;
```

```
                break;
```

```
            }
```

```

}

int free_count = count_free_frames();

printf("Page %d: free=%d\n", page, free_count);

if (free_count < LOW) {      // free_count가 LOW보다 작을 때 page daemon을 실행한다.
    run_page_daemon();
}
}
exit(0);
}

```

LOW=2 HIGH=5

Page 6: free=2

Page 7: free=1

Daemon: evicted=4, free=5

Page 8: free=4

Page 9: free=3

연습문제1

설정 값: TOTAL = 6, LOW = 2, HIGH = 4

초기 메모리 상태 (frames): {10, 11, FREE, FREE, FREE, FREE} (현재 빈 프레임 수: 4개)

페이지 요청 리스트 (requests): 12, 13, 14, 15

정답

LOW=2 HIGH=4

Page 12: free=3

Page 13: free=2

Page 14: free=1

Daemon: evicted=3, free=4

Page 15: free=3

연습문제2

설정 값: TOTAL = 8, LOW = 3, HIGH = 6

초기 메모리 상태 (frames): {1, 2, 3, 4, 5, 6, FREE, FREE} (현재 빈 프레임 수: 2개)

페이지 요청 리스트 (requests): 7, 8, 9

정답

LOW=3 HIGH=6

Page 7: free=1

Daemon: evicted=5, free=6

Page 8: free=5

Page 9: free=4